

effectiveness depends on the choice of synchronizing set. The sets should be chosen so that the parser recovers quickly from errors that are likely to occur in practice. Some heuristics are as follows:

1. As a starting point, we can place all symbols in $\text{FOLLOW}(A)$ into the synchronizing set for nonterminal A . If we skip tokens until an element of $\text{FOLLOW}(A)$ is seen and pop A from the stack, it is likely that parsing can continue.
2. It is not enough to use $\text{FOLLOW}(A)$ as the synchronizing set for A . For example, if semicolons terminate statements, as in C, then keywords that begin statements may not appear in the FOLLOW set of the nonterminal generating expressions. A missing semicolon after an assignment may therefore result in the keyword beginning the next statement being skipped. Often, there is a hierarchical structure on constructs in a language; e.g., expressions appear within statements, which appear within blocks, and so on. We can add to the synchronizing set of a lower construct the symbols that begin higher constructs. For example, we might add keywords that begin statements to the synchronizing sets for the nonterminals generating expressions.
3. If we add symbols in $\text{FIRST}(A)$ to the synchronizing set for nonterminal A , then it may be possible to resume parsing according to A if a symbol in $\text{FIRST}(A)$ appears in the input.
4. If a nonterminal can generate the empty string, then the production deriving ϵ can be used as a default. Doing so may postpone some error detection, but cannot cause an error to be missed. This approach reduces the number of nonterminals that have to be considered during error recovery.
5. If a terminal on top of the stack cannot be matched, a simple idea is to pop the terminal, issue a message saying that the terminal was inserted, and continue parsing. In effect, this approach takes the synchronizing set of a token to consist of all other tokens.

Example 4.20. Using FOLLOW and FIRST symbols as synchronizing tokens works reasonably well when expressions are parsed according to grammar (4.11). The parsing table for this grammar in Fig. 4.15 is repeated in Fig. 4.18, with “synch” indicating synchronizing tokens obtained from the FOLLOW set of the nonterminal in question. The FOLLOW sets for the nonterminal are obtained from Example 4.17.

The table in Fig. 4.18 is to be used as follows. If the parser looks up entry $M[A, a]$ and finds that it is blank, then the input symbol a is skipped. If the entry is synch, then the nonterminal on top of the stack is popped in an attempt to resume parsing. If a token on top of the stack does not match the input symbol, then we pop the token from the stack, as mentioned above.

On the erroneous input $)\text{id} * + \text{id}$ the parser and error recovery mechanism of Fig. 4.18 behave as in Fig. 4.19. □

NONTER-MINAL	INPUT SYMBOL					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$	synch	synch
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$	synch		$T \rightarrow FT'$	synch	synch
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow \text{id}$	synch	synch	$F \rightarrow (E)$	synch	synch

Fig. 4.18. Synchronizing tokens added to parsing table of Fig. 4.15.

STACK	INPUT	REMARK
$\$E$	) id * + id \$	error, skip)
$\$E$	id * + id \$	id is in FIRST(E)
$\$E'T$	id * + id \$	
$\$E'T'F$	id * + id \$	
$\$E'T'\text{id}$	id * + id \$	
$\$E'T'$	* + id \$	
$\$E'T'F*$	* + id \$	
$\$E'T'F$	+ id \$	error, $M[F, +] = \text{synch}$
$\$E'T'$	+ id \$	F has been popped
$\$E'$	+ id \$	
$\$E'T +$	+ id \$	
$\$E'T$	id \$	
$\$E'T'F$	id \$	
$\$E'T'\text{id}$	id \$	
$\$E'T'$	\$	
$\$E'$	\$	
\$	\$	

Fig. 4.19. Parsing and error recovery moves made by predictive parser.

The above discussion of panic-mode recovery does not address the important issue of error messages. In general, informative error messages have to be supplied by the compiler designer.

Phrase-level recovery. Phrase-level recovery is implemented by filling in the blank entries in the predictive parsing table with pointers to error routines. These routines may change, insert, or delete symbols on the input and issue appropriate error messages. They may also pop from the stack. It is questionable whether we should permit alteration of stack symbols or the pushing of new symbols onto the stack, since then the steps carried out by the parser might not correspond to the derivation of any word in the language at all. In

any event, we must be sure that there is no possibility of an infinite loop. Checking that any recovery action eventually results in an input symbol being consumed (or the stack being shortened if the end of the input has been reached) is a good way to protect against such loops.

4.5 BOTTOM-UP PARSING

In this section, we introduce a general style of bottom-up syntax analysis, known as shift-reduce parsing. An easy-to-implement form of shift-reduce parsing, called operator-precedence parsing, is presented in Section 4.6. A much more general method of shift-reduce parsing, called LR parsing, is discussed in Section 4.7. LR parsing is used in a number of automatic parser generators.

Shift-reduce parsing attempts to construct a parse tree for an input string beginning at the leaves (the bottom) and working up towards the root (the top). We can think of this process as one of "reducing" a string w to the start symbol of a grammar. At each *reduction* step a particular substring matching the right side of a production is replaced by the symbol on the left of that production, and if the substring is chosen correctly at each step, a rightmost derivation is traced out in reverse.

Example 4.21. Consider the grammar

$$\begin{aligned} S &\rightarrow aABe \\ A &\rightarrow Abc \mid b \\ B &\rightarrow d \end{aligned}$$

The sentence $abbcd$ can be reduced to S by the following steps:

$abbcd$
 $aAbcd$
 $aAde$
 $aABe$
 S

We scan $abbcd$ looking for a substring that matches the right side of some production. The substrings b and d qualify. Let us choose the leftmost b and replace it by A , the left side of the production $A \rightarrow b$; we thus obtain the string $aAbcd$. Now the substrings Abc , b , and d match the right side of some production. Although b is the leftmost substring that matches the right side of some production, we choose to replace the substring Abc by A , the left side of the production $A \rightarrow Abc$. We now obtain $aAde$. Then replacing d by B , the left side of the production $B \rightarrow d$, we obtain $aABe$. We can now replace this entire string by S . Thus, by a sequence of four reductions we are able to reduce $abbcd$ to S . These reductions, in fact, trace out the following rightmost derivation in reverse:

$$S \xRightarrow{rm} aABe \xRightarrow{rm} aAde \xRightarrow{rm} aAbcd \xRightarrow{rm} abbcd$$

□

Handles

Informally, a “handle” of a string is a substring that matches the right side of a production, and whose reduction to the nonterminal on the left side of the production represents one step along the reverse of a rightmost derivation. In many cases the leftmost substring β that matches the right side of some production $A \rightarrow \beta$ is not a handle, because a reduction by the production $A \rightarrow \beta$ yields a string that cannot be reduced to the start symbol. In Example 4.21, if we replaced b by A in the second string $aAbcde$ we would obtain the string $aAAcde$ that cannot be subsequently reduced to S . For this reason, we must give a more precise definition of a handle.

Formally, a *handle* of a right-sentential form γ is a production $A \rightarrow \beta$ and a position of γ where the string β may be found and replaced by A to produce the previous right-sentential form in a rightmost derivation of γ . That is, if $S \xRightarrow{*}_{rm} \alpha A w \xRightarrow{1}_{rm} \alpha \beta w$, then $A \rightarrow \beta$ in the position following α is a handle of $\alpha \beta w$. The string w to the right of the handle contains only terminal symbols. Note we say “a handle” rather than “the handle” because the grammar could be ambiguous, with more than one rightmost derivation of $\alpha \beta w$. If a grammar is unambiguous, then every right-sentential form of the grammar has exactly one handle.

In the example above, $abbcd$ is a right-sentential form whose handle is $A \rightarrow b$ at position 2. Likewise, $aAbcde$ is a right-sentential form whose handle is $A \rightarrow Abc$ at position 2. Sometimes we say “the substring β is a handle of $\alpha \beta w$ ” if the position of β and the production $A \rightarrow \beta$ we have in mind are clear.

Figure 4.20 portrays the handle $A \rightarrow \beta$ in the parse tree of a right-sentential form $\alpha \beta w$. The handle represents the leftmost complete subtree consisting of a node and all its children. In Fig. 4.20, A is the bottommost leftmost interior node with all its children in the tree. Reducing β to A in $\alpha \beta w$ can be thought of as “pruning the handle,” that is, removing the children of A from the parse tree.

Example 4.22. Consider the following grammar

- (1) $E \rightarrow E + E$
 - (2) $E \rightarrow E * E$
 - (3) $E \rightarrow (E)$
 - (4) $E \rightarrow \text{id}$
- (4.16)

and the rightmost derivation

$$\begin{aligned}
 E &\xRightarrow{*}_{rm} E + E \\
 &\xRightarrow{*}_{rm} E + E * E \\
 &\xRightarrow{*}_{rm} E + E * \underline{\text{id}_3} \\
 &\xRightarrow{*}_{rm} E + \underline{\text{id}_2} * \text{id}_3 \\
 &\xRightarrow{*}_{rm} \underline{\text{id}_1} + \text{id}_2 * \text{id}_3
 \end{aligned}$$

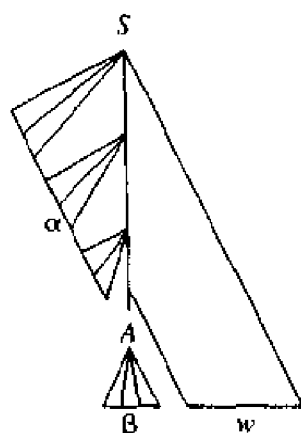


Fig. 4.20. The handle $A \rightarrow \beta$ in the parse tree for $\alpha\beta w$.

We have subscripted the **id**'s for notational convenience and underlined a handle of each right-sentential form. For example, id_1 is a handle of the right-sentential form $\text{id}_1 + \text{id}_2 * \text{id}_3$ because **id** is the right side of the production $E \rightarrow \text{id}$, and replacing id_1 by E produces the previous right-sentential form $E + \text{id}_2 * \text{id}_3$. Note that the string appearing to the right of a handle contains only terminal symbols.

Because grammar (4.16) is ambiguous, there is another rightmost derivation of the same string:

$$\begin{aligned}
 E &\xRightarrow{rm} \underline{E * E} \\
 &\xRightarrow{rm} E * \underline{\text{id}_3} \\
 &\xRightarrow{rm} \underline{E + E} * \text{id}_3 \\
 &\xRightarrow{rm} E + \underline{\text{id}_2 * \text{id}_3} \\
 &\xRightarrow{rm} \underline{\text{id}_1} + \text{id}_2 * \text{id}_3
 \end{aligned}$$

Consider the right sentential form $E + E * \text{id}_3$. In this derivation, $E + E$ is a handle of $E + E * \text{id}_3$ whereas id_3 by itself is a handle of this same right-sentential form according to the derivation above.

The two rightmost derivations in this example are analogs of the two leftmost derivations in Example 4.6. The first derivation gives $*$ a higher precedence than $+$, whereas the second gives $+$ the higher precedence. $\square$

Handle Pruning

A rightmost derivation in reverse can be obtained by "handle pruning." That is, we start with a string of terminals w that we wish to parse. If w is a sentence of the grammar at hand, then $w = \gamma_n$, where γ_n is the n th right-sentential form of some as yet unknown rightmost derivation.

$$S = \gamma_0 \xRightarrow{rm} \gamma_1 \xRightarrow{rm} \gamma_2 \xRightarrow{rm} \cdots \xRightarrow{rm} \gamma_{n-1} \xRightarrow{rm} \gamma_n = w.$$

To reconstruct this derivation in reverse order, we locate the handle β_n in γ_n and replace β_n by the left side of some production $A_n \rightarrow \beta_n$ to obtain the $(n - 1)$ st right-sentential form γ_{n-1} . Note that we do not yet know how handles are to be found, but we shall see methods of doing so shortly.

We then repeat this process. That is, we locate the handle β_{n-1} in γ_{n-1} and reduce this handle to obtain the right-sentential form γ_{n-2} . If by continuing this process we produce a right-sentential form consisting only of the start symbol S , then we halt and announce successful completion of parsing. The reverse of the sequence of productions used in the reductions is a rightmost derivation for the input string.

Example 4.23. Consider the grammar (4.16) of Example 4.22 and the input string $\text{id}_1 + \text{id}_2 * \text{id}_3$. The sequence of reductions shown in Fig. 4.21 reduces $\text{id}_1 + \text{id}_2 * \text{id}_3$ to the start symbol E . The reader should observe that the sequence of right-sentential forms in this example is just the reverse of the sequence in the first rightmost derivation in Example 4.22. □

RIGHT-SENTENTIAL FORM	HANDLE	REDUCING PRODUCTION
$\text{id}_1 + \text{id}_2 * \text{id}_3$	id_1	$E \rightarrow \text{id}$
$E + \text{id}_2 * \text{id}_3$	id_2	$E \rightarrow \text{id}$
$E + E * \text{id}_3$	id_3	$E \rightarrow \text{id}$
$E + E * E$	$E * E$	$E \rightarrow E * E$
$E + E$	$E + E$	$E \rightarrow E + E$
E		

Fig. 4.21. Reductions made by shift-reduce parser.

Stack Implementation of Shift-Reduce Parsing

There are two problems that must be solved if we are to parse by handle pruning. The first is to locate the substring to be reduced in a right-sentential form, and the second is to determine what production to choose in case there is more than one production with that substring on the right side. Before we get to these questions, let us first consider the type of data structures to use in a shift-reduce parser.

A convenient way to implement a shift-reduce parser is to use a stack to hold grammar symbols and an input buffer to hold the string w to be parsed. We use $\$$ to mark the bottom of the stack and also the right end of the input. Initially, the stack is empty, and the string w is on the input, as follows:

STACK	INPUT
$\$$	$w \$$

The parser operates by shifting zero or more input symbols onto the stack until a handle β is on top of the stack. The parser then reduces β to the left

side of the appropriate production. The parser repeats this cycle until it has detected an error or until the stack contains the start symbol and the input is empty:

STACK	INPUT
\$\$	\$

After entering this configuration, the parser halts and announces successful completion of parsing.

Example 4.24. Let us step through the actions a shift-reduce parser might make in parsing the input string $id_1 + id_2 * id_3$ according to grammar (4.16), using the first derivation of Example 4.22. The sequence is shown in Fig. 4.22. Note that because grammar (4.16) has two rightmost derivations for this input there is another sequence of steps a shift-reduce parser might take. $\square$

STACK	INPUT	ACTION
(1) \$	$id_1 + id_2 * id_3 \$$	shift
(2) $\$id_1$	$+ id_2 * id_3 \$$	reduce by $E \rightarrow id$
(3) $\$E$	$+ id_2 * id_3 \$$	shift
(4) $\$E +$	$id_2 * id_3 \$$	shift
(5) $\$E + id_2$	$* id_3 \$$	reduce by $E \rightarrow id$
(6) $\$E + E$	$* id_3 \$$	shift
(7) $\$E + E *$	$id_3 \$$	shift
(8) $\$E + E * id_3$	\$	reduce by $E \rightarrow id$
(9) $\$E + E * E$	\$	reduce by $E \rightarrow E * E$
(10) $\$E + E$	\$	reduce by $E \rightarrow E + E$
(11) $\$E$	\$	accept

Fig. 4.22. Configurations of shift-reduce parser on input $id_1 + id_2 * id_3$.

While the primary operations of the parser are shift and reduce, there are actually four possible actions a shift-reduce parser can make: (1) shift, (2) reduce, (3) accept, and (4) error.

1. In a *shift* action, the next input symbol is shifted onto the top of the stack.
2. In a *reduce* action, the parser knows the right end of the handle is at the top of the stack. It must then locate the left end of the handle within the stack and decide with what nonterminal to replace the handle.
3. In an *accept* action, the parser announces successful completion of parsing.
4. In an *error* action, the parser discovers that a syntax error has occurred and calls an error recovery routine.

There is an important fact that justifies the use of a stack in shift-reduce parsing: the handle will always eventually appear on top of the stack, never inside. This fact becomes obvious when we consider the possible forms of two successive steps in any rightmost derivation. These two steps can be of the form

$$\begin{aligned} (1) \quad S &\xRightarrow{*}_{rm} \alpha Az \xRightarrow{*}_{rm} \alpha \beta B y z \xRightarrow{*}_{rm} \alpha \beta \gamma y z \\ (2) \quad S &\xRightarrow{*}_{rm} \alpha B x A z \xRightarrow{*}_{rm} \alpha B x y z \xRightarrow{*}_{rm} \alpha \gamma x y z \end{aligned}$$

In case (1), A is replaced by $\beta B y$, and then the rightmost nonterminal B in that right side is replaced by γ . In case (2), A is again replaced first, but this time the right side is a string y of terminals only. The next rightmost nonterminal B will be somewhere to the left of y .

Let us consider case (1) in reverse, where a shift-reduce parser has just reached the configuration

STACK	INPUT
\$ $\alpha\beta\gamma$	yz\$

The parser now reduces the handle γ to B to reach the configuration

STACK	INPUT
\$ $\alpha\beta B$	yz\$

Since B is the rightmost nonterminal in $\alpha\beta B y z$, the right end of the handle of $\alpha\beta B y z$ cannot occur inside the stack. The parser can therefore shift the string y onto the stack to reach the configuration

STACK	INPUT
\$ $\alpha\beta B y$	z\$

in which $\beta B y$ is the handle, and it gets reduced to A .

In case (2), in configuration

STACK	INPUT
\$ $\alpha\gamma$	xyz\$

the handle γ is on top of the stack. After reducing the handle γ to B , the parser can shift the string xy to get the next handle y on top of the stack:

STACK	INPUT
\$ $\alpha B x y$	z\$

Now the parser reduces y to A .

In both cases, after making a reduction the parser had to shift zero or more symbols to get the next handle onto the stack. It never had to go into the stack to find the handle. It is this aspect of handle pruning that makes a stack a particularly convenient data structure for implementing a shift-reduce parser. We still must explain how choices of action are to be made so the shift-reduce parser works correctly. Operator precedence and LR parsers are two such techniques that we shall discuss shortly.

Viable Prefixes

The set of prefixes of right sentential forms that can appear on the stack of a shift-reduce parser are called *viable prefixes*. An equivalent definition of a viable prefix is that it is a prefix of a right-sentential form that does not continue past the right end of the rightmost handle of that sentential form. By this definition, it is always possible to add terminal symbols to the end of a viable prefix to obtain a right-sentential form. Therefore, there is apparently no error as long as the portion of the input seen to a given point can be reduced to a viable prefix.

Conflicts During Shift-Reduce Parsing

There are context-free grammars for which shift-reduce parsing cannot be used. Every shift-reduce parser for such a grammar can reach a configuration in which the parser, knowing the entire stack contents and the next input symbol, cannot decide whether to shift or to reduce (a *shift/reduce conflict*), or cannot decide which of several reductions to make (a *reduce/reduce conflict*). We now give some examples of syntactic constructs that give rise to such grammars. Technically, these grammars are not in the $LR(k)$ class of grammars defined in Section 4.7; we refer to them as non-LR grammars. The k in $LR(k)$ refers to the number of symbols of lookahead on the input. Grammars used in compiling usually fall in the $LR(1)$ class, with one symbol lookahead.

Example 4.25. An ambiguous grammar can never be LR. For example, consider the dangling-else grammar (4.7) of Section 4.3:

$$\begin{array}{l} stmt \rightarrow \text{if } expr \text{ then } stmt \\ \quad | \text{ if } expr \text{ then } stmt \text{ else } stmt \\ \quad | \text{ other} \end{array}$$

If we have a shift-reduce parser in configuration

STACK	INPUT
· · · if <i>expr</i> then <i>stmt</i>	else · · · \$

we cannot tell whether *if expr then stmt* is the handle, no matter what appears below it on the stack. Here there is a shift/reduce conflict. Depending on what follows the *else* on the input, it might be correct to reduce *if expr then stmt* to *stmt*, or it might be correct to shift *else* and then to look for another *stmt* to complete the alternative *if expr then stmt else stmt*. Thus, we cannot tell whether to shift or reduce in this case, so the grammar is not $LR(1)$. More generally, no ambiguous grammar, as this one certainly is, can be $LR(k)$ for any k .

We should mention, however, that shift-reduce parsing can be easily adapted to parse certain ambiguous grammars, such as the if-then-else grammar above. When we construct such a parser for a grammar containing the two productions above, there will be a shift/reduce conflict; on *else*, either shift, or reduce by *stmt* $\rightarrow$ *if expr then stmt*. If we resolve the conflict in favor

of shifting, the parser will behave naturally. We discuss parsers for such ambiguous grammars in Section 4.8. $\square$

Another common cause of non-LR-ness occurs when we know we have a handle, but the stack contents and the next input symbol are not sufficient to determine which production should be used in a reduction. The next example illustrates this situation.

Example 4.26. Suppose we have a lexical analyzer that returns token **id** for all identifiers, regardless of usage. Suppose also that our language invokes procedures by giving their names, with parameters surrounded by parentheses, and that arrays are referenced by the same syntax. Since the translation of indices in array references and parameters in procedure calls are different, we want to use different productions to generate lists of actual parameters and indices. Our grammar might therefore have (among others) productions such as:

- (1) $stmt \rightarrow id(parameter_list)$
- (2) $stmt \rightarrow expr := expr$
- (3) $parameter_list \rightarrow parameter_list , parameter$
- (4) $parameter_list \rightarrow parameter$
- (5) $parameter \rightarrow id$
- (6) $expr \rightarrow id(expr_list)$
- (7) $expr \rightarrow id$
- (8) $expr_list \rightarrow expr_list , expr$
- (9) $expr_list \rightarrow expr$

A statement beginning with **A(I, J)** would appear as the token stream **id(id, id)** to the parser. After shifting the first three tokens onto the stack, a shift-reduce parser would be in configuration

STACK	INPUT
... id (id	, id) ...

It is evident that the **id** on top of the stack must be reduced, but by which production? The correct choice is production (5) if **A** is a procedure and production (7) if **A** is an array. The stack does not tell which; information in the symbol table obtained from the declaration of **A** has to be used.

One solution is to change the token **id** in production (1) to **procid** and to use a more sophisticated lexical analyzer that returns token **procid** when it recognizes an identifier which is the name of a procedure. Doing this would require the lexical analyzer to consult the symbol table before returning a token.

If we made this modification, then on processing **A(I, J)** the parser would be either in the configuration

STACK	INPUT
... procid (id	, id) ...

or in the configuration above. In the former case, we choose reduction by production (5); in the latter case by production (7). Notice how the symbol third from the top of the stack determines the reduction to be made, even though it is not involved in the reduction. Shift-reduce parsing can utilize information far down in the stack to guide the parse. $\square$

4.6 OPERATOR-PRECEDENCE PARSING

The largest class of grammars for which shift-reduce parsers can be built successfully – the LR grammars – will be discussed in Section 4.7. However, for a small but important class of grammars we can easily construct efficient shift-reduce parsers by hand. These grammars have the property (among other essential requirements) that no production right side is ϵ or has two adjacent nonterminals. A grammar with the latter property is called an *operator grammar*.

Example 4.27. The following grammar for expressions

$$\begin{aligned} E &\rightarrow EAE \mid (E) \mid -E \mid \text{id} \\ A &\rightarrow + \mid - \mid * \mid / \mid \uparrow \end{aligned}$$

is not an operator grammar, because the right side EAE has two (in fact three) consecutive nonterminals. However, if we substitute for A each of its alternatives, we obtain the following operator grammar:

$$E \rightarrow E+E \mid E-E \mid E * E \mid E / E \mid E \uparrow E \mid (E) \mid -E \mid \text{id} \quad (4.17)$$

We now describe an easy-to-implement parsing technique called operator-precedence parsing. Historically, the technique was first described as a manipulation on tokens without any reference to an underlying grammar. In fact, once we finish building an operator-precedence parser from a grammar, we may effectively ignore the grammar, using the nonterminals on the stack only as placeholders for attributes associated with the nonterminals.

As a general parsing technique, operator-precedence parsing has a number of disadvantages. For example, it is hard to handle tokens like the minus sign, which has two different precedences (depending on whether it is unary or binary). Worse, since the relationship between a grammar for the language being parsed and the operator-precedence parser itself is tenuous, one cannot always be sure the parser accepts exactly the desired language. Finally, only a small class of grammars can be parsed using operator-precedence techniques.

Nevertheless, because of its simplicity, numerous compilers using operator-precedence parsing techniques for expressions have been built successfully. Often these parsers use recursive descent, described in Section 4.4, for statements and higher-level constructs. Operator-precedence parsers have even been built for entire languages.

In operator-precedence parsing, we define three disjoint *precedence relations*, $<$, $\doteq$, and $>$, between certain pairs of terminals. These precedence relations guide the selection of handles and have the following meanings:

RELATION	MEANING
$a < \cdot b$	a "yields precedence to" b
$a \dot{=} b$	a "has the same precedence as" b
$a \cdot > b$	a "takes precedence over" b

We should caution the reader that while these relations may appear similar to the arithmetic relations "less than," "equal to," and "greater than," the precedence relations have quite different properties. For example, we could have $a < \cdot b$ and $a \cdot > b$ for the same language, or we might have none of $a < \cdot b$, $a \dot{=} b$, and $a \cdot > b$ holding for some terminals a and b .

There are two common ways of determining what precedence relations should hold between a pair of terminals. The first method we discuss is intuitive and is based on the traditional notions of associativity and precedence of operators. For example, if $*$ is to have higher precedence than $+$, we make $+ < \cdot *$ and $* \cdot > +$. This approach will be seen to resolve the ambiguities of grammar (4.17), and it enables us to write an operator-precedence parser for it (although the unary minus sign causes problems).

The second method of selecting operator-precedence relations is first to construct an unambiguous grammar for the language, a grammar that reflects the correct associativity and precedence in its parse trees. This job is not difficult for expressions; the syntax of expressions in Section 2.2 provides the paradigm. For the other common source of ambiguity, the dangling *else*, grammar (4.9) is a useful model. Having obtained an unambiguous grammar, there is a mechanical method for constructing operator-precedence relations from it. These relations may not be disjoint, and they may parse a language other than that generated by the grammar, but with the standard sorts of arithmetic expressions, few problems are encountered in practice. We shall not discuss this construction here; see Aho and Ullman [1972b].

Using Operator-Precedence Relations

The intention of the precedence relations is to delimit the handle of a right-sentential form, with $< \cdot$ marking the left end, $\dot{=}$ appearing in the interior of the handle, and $\cdot >$ marking the right end. To be more precise, suppose we have a right-sentential form of an operator grammar. The fact that no adjacent nonterminals appear on the right sides of productions implies that no right-sentential form will have two adjacent nonterminals either. Thus, we may write the right-sentential form as $\beta_0 a_1 \beta_1 \cdots a_n \beta_n$, where each β_i is either ϵ (the empty string) or a single nonterminal, and each a_i is a single terminal.

Suppose that between a_i and a_{i+1} exactly one of the relations $< \cdot$, $\dot{=}$, and $\cdot >$ holds. Further, let us use $\$$ to mark each end of the string, and define $\$ < \cdot b$ and $b \cdot > \$$ for all terminals b . Now suppose we remove the nonterminals from the string and place the correct relation $< \cdot$, $\dot{=}$, or $\cdot >$, between each

pair of terminals and between the endmost terminals and the \$'s marking the ends of the string. For example, suppose we initially have the right-sentential form $\text{id} + \text{id} * \text{id}$ and the precedence relations are those given in Fig. 4.23. These relations are some of those that we would choose to parse according to grammar (4.17).

	id	+	*	\$
id		$\cdot >$	$\cdot >$	$\cdot >$
+	$< \cdot$	$\cdot >$	$< \cdot$	$\cdot >$
*	$< \cdot$	$\cdot >$	$\cdot >$	$\cdot >$
\$	$< \cdot$	$< \cdot$	$< \cdot$	

Fig. 4.23. Operator-precedence relations.

Then the string with the precedence relations inserted is:

$$\$ < \cdot \text{id} \cdot > + < \cdot \text{id} \cdot > * < \cdot \text{id} \cdot > \$ \quad (4.18)$$

For example, $< \cdot$ is inserted between the leftmost \$ and id since $< \cdot$ is the entry in row \$ and column id . The handle can be found by the following process.

1. Scan the string from the left end until the first $\cdot >$ is encountered. In (4.18) above, this occurs between the first id and $+$.
2. Then scan backwards (to the left) over any $=$'s until a $< \cdot$ is encountered. In (4.18), we scan backwards to \$.
3. The handle contains everything to the left of the first $\cdot >$ and to the right of the $< \cdot$ encountered in step (2), including any intervening or surrounding nonterminals. (The inclusion of surrounding nonterminals is necessary so that two adjacent nonterminals do not appear in a right-sentential form.) In (4.18), the handle is the first id .

If we are dealing with grammar (4.17), we then reduce id to E . At this point we have the right-sentential form $E + \text{id} * \text{id}$. After reducing the two remaining id 's to E by the same steps, we obtain the right-sentential form $E + E * E$. Consider now the string $\$ * \$$ obtained by deleting the nonterminals. Inserting the precedence relations, we get

$$\$ < \cdot + < \cdot * \cdot > \$$$

indicating that the left end of the handle lies between $+$ and $*$ and the right end between $*$ and $\$$. These precedence relations indicate that, in the right-sentential form $E + E * E$, the handle is $E * E$. Note how the E 's surrounding the $*$ become part of the handle.

Since the nonterminals do not influence the parse, we need not worry about distinguishing among them. A single marker "nonterminal" can be kept on

the stack of a shift-reduce parser to indicate placeholders for attribute values.

It may appear from the discussion above that the entire right-sentential form must be scanned at each step to find the handle. Such is not the case if we use a stack to store the input symbols already seen and if the precedence relations are used to guide the actions of a shift-reduce parser. If the precedence relation $<\cdot$ or $\doteq$ holds between the topmost terminal symbol on the stack and the next input symbol, the parser shifts; it has not yet found the right end of the handle. If the relation $\cdot>$ holds, a reduction is called for. At this point the parser has found the right end of the handle, and the precedence relations can be used to find the left end of the handle in the stack.

If no precedence relation holds between a pair of terminals (indicated by a blank entry in Fig. 4.23), then a syntactic error has been detected and an error recovery routine must be invoked, as discussed later in this section. The above ideas can be formalized by the following algorithm.

Algorithm 4.5. Operator-precedence parsing algorithm.

Input. An input string w and a table of precedence relations.

Output. If w is well formed, a *skeletal* parse tree, with a placeholder nonterminal E labeling all interior nodes; otherwise, an error indication.

Method. Initially, the stack contains $\$$ and the input buffer the string $w\$$. To parse, we execute the program of Fig. 4.24. $\square$

```

(1) set  $ip$  to point to the first symbol of  $w\$$ ;
(2) repeat forever
(3)   if  $\$$  is on top of the stack and  $ip$  points to  $\$$  then
(4)     return
      else begin
(5)       let  $a$  be the topmost terminal symbol on the stack
          and let  $b$  be the symbol pointed to by  $ip$ ;
(6)       if  $a <\cdot b$  or  $a \doteq b$  then begin
(7)         push  $b$  onto the stack;
(8)         advance  $ip$  to the next input symbol;
      end;
(9)       else if  $a \cdot > b$  then          /* reduce */
(10)        repeat
(11)          pop the stack
(12)        until the top stack terminal is related by  $<\cdot$ 
            to the terminal most recently popped
(13)       else error()
end

```

Fig. 4.24. Operator-precedence parsing algorithm.

Operator-Precedence Relations from Associativity and Precedence

We are always free to create operator-precedence relations any way we see fit and hope that the operator-precedence parsing algorithm will work correctly when guided by them. For a language of arithmetic expressions such as that generated by grammar (4.17) we can use the following heuristic to produce a proper set of precedence relations. Note that grammar (4.17) is ambiguous, and right-sentential forms could have many handles. Our rules are designed to select the "proper" handles to reflect a given set of associativity and precedence rules for binary operators.

1. If operator θ_1 has higher precedence than operator θ_2 , make $\theta_1 \cdot > \theta_2$ and $\theta_2 < \cdot \theta_1$. For example, if $*$ has higher precedence than $+$, make $* \cdot > +$ and $+ < \cdot *$. These relations ensure that, in an expression of the form $E + E * E + E$, the central $E * E$ is the handle that will be reduced first.
2. If θ_1 and θ_2 are operators of equal precedence (they may in fact be the same operator), then make $\theta_1 \cdot > \theta_2$ and $\theta_2 \cdot > \theta_1$ if the operators are left-associative, or make $\theta_1 < \cdot \theta_2$ and $\theta_2 < \cdot \theta_1$ if they are right-associative. For example, if $+$ and $-$ are left-associative, then make $+ \cdot > +$, $+ \cdot > -$, $- \cdot > -$, and $- \cdot > +$. If $\uparrow$ is right associative, then make $\uparrow < \cdot \uparrow$. These relations ensure that $E - E + E$ will have handle $E - E$ selected and $E \uparrow E \uparrow E$ will have the last $E \uparrow E$ selected.
3. Make $\theta < \cdot \text{id}$, $\text{id} \cdot > \theta$, $\theta < \cdot ($, $(< \cdot \theta$, $) \cdot > \theta$, $\theta \cdot >)$, $\theta \cdot > \$$, and $\$ < \cdot \theta$ for all operators θ . Also, let

$$\begin{array}{lll}
 (\doteq) & \$ < \cdot (& \$ < \cdot \text{id} \\
 (< \cdot (& \text{id} \cdot > \$ &) \cdot > \$ \\
 (< \cdot \text{id} & \text{id} \cdot >) &) \cdot >)
 \end{array}$$

These rules ensure that both id and (E) will be reduced to E . Also, $\$$ serves as both the left and right endmarker, causing handles to be found between $\$$'s wherever possible.

Example 4.28. Figure 4.25 contains the operator-precedence relations for grammar (4.17), assuming

1. $\uparrow$ is of highest precedence and right-associative,
2. $*$ and $/$ are of next highest precedence and left-associative, and
3. $+$ and $-$ are of lowest precedence and left-associative,

(Blanks denote error entries.) The reader should try out the table to see that it works correctly, ignoring problems with unary minus for the moment. Try the table on the input $\text{id} * (\text{id} \uparrow \text{id}) - \text{id} / \text{id}$, for example. $\square$

	+	-	*	/	↑	id	(	)	\$
+	·>	·>	<·	<·	<·	<·	<·	·>	·>
-	·>	·>	<·	<·	<·	<·	<·	·>	·>
*	·>	·>	·>	·>	<·	<·	<·	·>	·>
/	·>	·>	·>	·>	<·	<·	<·	·>	·>
↑	·>	·>	·>	·>	<·	<·	<·	·>	·>
id	·>	·>	·>	·>	·>			·>	·>
(	<·	<·	<·	<·	<·	<·	<·	=	
)	·>	·>	·>	·>	·>			·>	·>
\$	<·	<·	<·	<·	<·	<·	<·		

Fig. 4.25. Operator-precedence relations.

Handling Unary Operators

If we have a unary operator such as $\neg$ (logical negation), which is not also a binary operator, we can incorporate it into the above scheme for creating operator-precedence relations. Supposing $\neg$ to be a unary prefix operator, we make $\theta < \cdot \neg$ for any operator θ , whether unary or binary. We make $\neg \cdot > \theta$ if $\neg$ has higher precedence than θ and $\neg < \cdot \theta$ if not. For example, if $\neg$ has higher precedence than $\&$, and $\&$ is left-associative, we would group $E \& \neg E \& E$ as $(E \& (\neg E)) \& E$, by these rules. The rule for unary postfix operators is analogous.

The situation changes when we have an operator like the minus sign $-$ that is both unary prefix and binary infix. Even if we give unary and binary minus the same precedence, the table of Fig. 4.25 will fail to parse strings like $\text{id} * - \text{id}$ correctly. The best approach in this case is to use the lexical analyzer to distinguish between unary and binary minus, by having it return a different token when it sees unary minus. Unfortunately, the lexical analyzer cannot use lookahead to distinguish the two; it must remember the previous token. In Fortran, for example, a minus sign is unary if the previous token was an operator, a left parenthesis, a comma, or an assignment symbol.

Precedence Functions

Compilers using operator-precedence parsers need not store the table of precedence relations. In most cases, the table can be encoded by two *precedence functions* f and g that map terminal symbols to integers. We attempt to select f and g so that, for symbols a and b ,

1. $f(a) < g(b)$ whenever $a < \cdot b$,
2. $f(a) = g(b)$ whenever $a \doteq b$, and
3. $f(a) > g(b)$ whenever $a \cdot > b$.

Thus the precedence relation between a and b can be determined by a

numerical comparison between $f(a)$ and $g(b)$. Note, however, that error entries in the precedence matrix are obscured, since one of (1), (2), or (3) holds no matter what $f(a)$ and $g(b)$ are. The loss of error detection capability is generally not considered serious enough to prevent the using of precedence functions where possible; errors can still be caught when a reduction is called for and no handle can be found.

Not every table of precedence relations has precedence functions to encode it, but in practical cases the functions usually exist.

Example 4.29. The precedence table of Fig. 4.25 has the following pair of precedence functions,

	+	-	*	/	↑	(	)	id	\$
f	2	2	4	4	4	0	6	6	0
g	1	1	3	3	5	5	0	5	0

For example, $* < \cdot \text{id}$, and $f(*) < g(\text{id})$. Note that $f(\text{id}) > g(\text{id})$ suggests that $\text{id} \cdot > \text{id}$; but, in fact, no precedence relation holds between id and id . Other error entries in Fig. 4.25 are similarly replaced by one or another precedence relation. $\square$

A simple method for finding precedence functions for a table, if such functions exist, is the following.

Algorithm 4.6. Constructing precedence functions.

Input. An operator precedence matrix.

Output. Precedence functions representing the input matrix, or an indication that none exist.

Method.

1. Create symbols f_a and g_a for each a that is a terminal or $\$$.
2. Partition the created symbols into as many groups as possible, in such a way that if $a \doteq b$, then f_a and g_b are in the same group. Note that we may have to put symbols in the same group even if they are not related by $\doteq$. For example, if $a \doteq b$ and $c \doteq b$, then f_a and f_c must be in the same group, since they are both in the same group as g_b . If, in addition, $c \doteq d$, then f_a and g_d are in the same group even though $a \doteq d$ may not hold.
3. Create a directed graph whose nodes are the groups found in (2). For any a and b , if $a < \cdot b$, place an edge from the group of g_b to the group of f_a . If $a \cdot > b$, place an edge from the group of f_a to that of g_b . Note that an edge or path from f_a to g_b means that $f(a)$ must exceed $g(b)$; a path from g_b to f_a means that $g(b)$ must exceed $f(a)$.
4. If the graph constructed in (3) has a cycle, then no precedence functions exist. If there are no cycles, let $f(a)$ be the length of the longest path

beginning at the group of f_a ; let $g(a)$ be the length of the longest path from the group of g_a . $\square$

Example 4.30. Consider the matrix of Fig. 4.23. There are no $\doteq$ relationships, so each symbol is in a group by itself. Figure 4.26 shows the graph constructed using Algorithm 4.6.

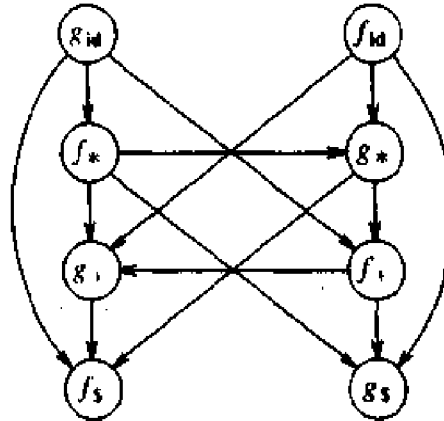


Fig. 4.26. Graph representing precedence functions.

There are no cycles, so precedence functions exist. As $f_\$$ and $g_\$$ have no out-edges, $f(\$) = g(\$) = 0$. The longest path from g_+ has length 1, so $g(+) = 1$. There is a path from g_{id} to f_* to g_* to f_+ to g_+ to $f_\$$, so $g(id) = 5$. The resulting precedence functions are:

	+	*	id	\$
f	2	4	4	0
g	1	3	5	0

$\square$

Error Recovery in Operator-Precedence Parsing

There are two points in the parsing process at which an operator-precedence parser can discover syntactic errors:

1. If no precedence relation holds between the terminal on top of the stack and the current input.¹
2. If a handle has been found, but there is no production with this handle as a right side.

Recall that the operator-precedence parsing algorithm (Algorithm 4.5) appears to reduce handles composed of terminals only. However, while nonterminals

¹ In compilers using precedence functions to represent the precedence tables, this source of error detection may be unavailable.

are treated anonymously, they still have places held for them on the parsing stack. Thus when we talk in (2) above about a handle matching a production's right side, we mean that the terminals are the same and the positions occupied by nonterminals are the same.

We should observe that, besides (1) and (2) above, there are no other points at which errors could be detected. When scanning down the stack to find the left end of the handle in steps (10-12) of Fig. 4.24, the operator-precedence parsing algorithm, we are sure to find a $<\cdot$ relation, since \$ marks the bottom of stack and is related by $<\cdot$ to any symbol that could appear immediately above it on the stack. Note also that we never allow adjacent symbols on the stack in Fig. 4.24 unless they are related by $<\cdot$ or $\doteq$. Thus steps (10-12) must succeed in making a reduction.

Just because we find a sequence of symbols $a <\cdot b_1 \doteq b_2 \doteq \cdots \doteq b_k$ on the stack, however, does not mean that $b_1 b_2 \cdots b_k$ is the string of terminal symbols on the right side of some production. We did not check for this condition in Fig. 4.24, but we clearly can do so, and in fact we must do so if we wish to associate semantic rules with reductions. Thus we have an opportunity to detect errors in Fig. 4.24, modified at steps (10-12) to determine what production is the handle in a reduction.

Handling Errors During Reductions

We may divide the error detection and recovery routine into several pieces. One piece handles errors of type (2). For example, this routine might pop symbols off the stack just as in steps (10-12) of Fig. 4.24. However, as there is no production to reduce by, no semantic actions are taken; a diagnostic message is printed instead. To determine what the diagnostic should say, the routine handling case (2) must decide what production the right side being popped "looks like." For example, suppose abc is popped, and there is no production right side consisting of a , b and c together with zero or more nonterminals. Then we might consider if deletion of one of a , b , and c yields a legal right side (nonterminals omitted). For example, if there were a right side $aEcE$, we might issue the diagnostic

illegal b on line (line containing b)

We might also consider changing or inserting a terminal. Thus if $abEdc$ were a right side, we might issue a diagnostic

missing d on line (line containing c)

We may also find that there is a right side with the proper sequence of terminals, but the wrong pattern of nonterminals. For example, if abc is popped off the stack with no intervening or surrounding nonterminals, and abc is not a right side but $aEbc$ is, we might issue a diagnostic

missing E on line (line containing b)

Here E stands for an appropriate syntactic category represented by nonterminal E . For example, if a , b , or c is an operator, we might say "expression;" if a is a keyword like *if*, we might say "conditional."

In general, the difficulty of determining appropriate diagnostics when no legal right side is found depends upon whether there are a finite or infinite number of possible strings that could be popped in lines (10-12) of Fig. 4.24. Any such string $b_1 b_2 \cdots b_k$ must have $\doteq$ relations holding between adjacent symbols, so $b_1 \doteq b_2 \doteq \cdots \doteq b_k$. If an operator precedence table tells us that there are only a finite number of sequences of terminals related by $\doteq$, then we can handle these strings on a case-by-case basis. For each such string x we can determine in advance a minimum-distance legal right side y and issue a diagnostic implying that x was found when y was intended.

It is easy to determine all strings that could be popped from the stack in steps (10-12) of Fig. 4.24. These are evident in the directed graph whose nodes represent the terminals, with an edge from a to b if and only if $a \doteq b$. Then the possible strings are the labels of the nodes along paths in this graph. Paths consisting of a single node are possible. However, in order for a path $b_1 b_2 \cdots b_k$ to be "poppable" on some input, there must be a symbol a (possibly $\$$) such that $a < b_1$. Call such a b_1 *initial*. Also, there must be a symbol c (possibly $\$$) such that $b_k > c$. Call b_k *final*. Only then could a reduction be called for and $b_1 b_2 \cdots b_k$ be the sequence of symbols popped. If the graph has a path from an initial to a final node containing a cycle, then there are an infinity of strings that might be popped; otherwise, there are only a finite number.

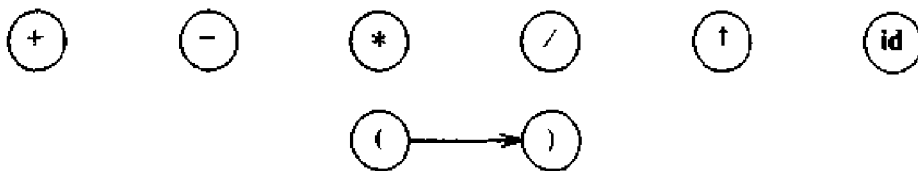


Fig. 4.27. Graph for precedence matrix of Fig. 4.25.

Example 4.31. Let us reconsider grammar (4.17):

$$E \rightarrow E + E \mid E - E \mid E * E \mid E / E \mid E \uparrow E \mid (E) \mid -E \mid \text{id}$$

The precedence matrix for this grammar was shown in Fig. 4.25, and its graph is given in Fig. 4.27. There is only one edge, because the only pair related by $\doteq$ is the left and right parenthesis. All but the right parenthesis are initial, and all but the left parenthesis are final. Thus the only paths from an initial to a final node are the paths $+$, $-$, $*$, $/$, id , and $\uparrow$ of length one, and the path from $($ to $)$ of length two. There are but a finite number, and each corresponds to the terminals of some production's right side in the grammar. Thus the error checker for reductions need only check that the proper set of

nonterminal markers appears among the terminal strings being reduced. Specifically, the checker does the following:

1. If $+$, $-$, $*$, $/$, or $\dagger$ is reduced, it checks that nonterminals appear on both sides. If not, it issues the diagnostic

missing operand

2. If id is reduced, it checks that there is no nonterminal to the right or left. If there is, it can warn

missing operator

3. If $(\)$ is reduced, it checks that there is a nonterminal between the parentheses. If not, it can say

no expression between parentheses

Also it must check that no nonterminal appears on either side of the parentheses. If one does, it issues the same diagnostic as in (2). $\square$

If there are an infinity of strings that may be popped, error messages cannot be tabulated on a case-by-case basis. We might use a general routine to determine whether some production right side is close (say distance 1 or 2, where distance is measured in terms of tokens, rather than characters, inserted, deleted, or changed) to the popped string and if so, issue a specific diagnostic on the assumption that that production was intended. If no production is close to the popped string, we can issue a general diagnostic to the effect that "something is wrong in the current line."

Handling Shift/Reduce Errors

We must now discuss the other way in which the operator-precedence parser detects errors. When consulting the precedence matrix to decide whether to shift or reduce (lines (6) and (9) of Fig. 4.24), we may find that no relation holds between the top stack symbol and the first input symbol. For example, suppose a and b are the two top stack symbols (b is at the top), c and d are the next two input symbols, and there is no precedence relation between b and c . To recover, we must modify the stack, input or both. We may change symbols, insert symbols onto the input or stack, or delete symbols from the input or stack. If we insert or change, we must be careful that we do not get into an infinite loop, where, for example, we perpetually insert symbols at the beginning of the input without being able to reduce or to shift any of the inserted symbols.

One approach that will assure us no infinite loops is to guarantee that after recovery the current input symbol can be shifted (if the current input is $\$,$ guarantee that no symbol is placed on the input, and the stack is eventually shortened). For example, given ab on the stack and cd on the input, if $a \leq c^2$

² We use $\leq$ to mean $<$ or $=$.

we might pop b from the stack. Another choice is to delete c from the input if $b \leq d$. A third choice is to find a symbol e such that $b \leq e \leq c$ and insert e in front of c on the input. More generally, we might insert a string of symbols such that

$$b \leq e_1 \leq e_2 \leq \dots \leq e_n \leq c$$

if a single symbol for insertion could not be found. The exact action chosen should reflect the compiler designer's intuition regarding what error is likely in each case.

For each blank entry in the precedence matrix we must specify an error-recovery routine; the same routine could be used in several places. Then when the parser consults the entry for a and b in step (6) of Fig. 4.24, and no precedence relation holds between a and b , it finds a pointer to the error-recovery routine for this error.

Example 4.32. Consider the precedence matrix of Fig. 4.25 again. In Fig. 4.28, we show the rows and columns of this matrix that have one or more blank entries, and we have filled in these blanks with the names of error-handling routines.

	id	(	)	\$
id	e3	e3	>	>
(	<	<	=	e4
)	e3	e3	>	>
\$	<	<	e2	e1

Fig. 4.28. Operator-precedence matrix with error entries.

The substance of these error handling routines is as follows:

- e1: /* called when whole expression is missing */
insert id onto the input
issue diagnostic: "missing operand"
- e2: /* called when expression begins with a right parenthesis */
delete) from the input
issue diagnostic: "unbalanced right parenthesis"
- e3: /* called when id or) is followed by id or (*/
insert + onto the input
issue diagnostic: "missing operator"
- e4: /* called when expression ends with a left parenthesis */
pop (from the stack
issue diagnostic: "missing right parenthesis"

Let us consider how this error-handling mechanism would treat the

erroneous input `id + )`. The first actions taken by the parser are to shift `id`, reduce it to `E` (we again use `E` for anonymous nonterminals on the stack), and then to shift the `+`. We now have configuration

STACK	INPUT
<code>\$E +</code>	<code>)\$</code>

Since `+` $\cdot$ `)` a reduction is called for, and the handle is `+`. The error checker for reductions is required to inspect for `E`'s to left and right. Finding one missing, it issues the diagnostic

missing operand

and does the reduction anyway.

Our configuration is now

<code>\$E</code>	<code>)\$</code>
------------------	------------------

There is no precedence relation between `$` and `)`, and the entry in Fig. 4.28 for this pair of symbols is `e2`. Routine `e2` causes diagnostic

unbalanced right parenthesis

to be printed and removes the right parenthesis from the input. We are now left with the final configuration for the parser.

<code>\$E</code>	<code>\$</code>	□
------------------	-----------------	---

4.7 LR PARSERS

This section presents an efficient, bottom-up syntax analysis technique that can be used to parse a large class of context-free grammars. The technique is called $LR(k)$ parsing; the "L" is for left-to-right scanning of the input, the "R" for constructing a rightmost derivation in reverse, and the k for the number of input symbols of lookahead that are used in making parsing decisions. When (k) is omitted, k is assumed to be 1. LR parsing is attractive for a variety of reasons.

- LR parsers can be constructed to recognize virtually all programming-language constructs for which context-free grammars can be written.
- The LR parsing method is the most general nonbacktracking shift-reduce parsing method known, yet it can be implemented as efficiently as other shift-reduce methods.
- The class of grammars that can be parsed using LR methods is a proper superset of the class of grammars that can be parsed with predictive parsers.
- An LR parser can detect a syntactic error as soon as it is possible to do so on a left-to-right scan of the input.

The principal drawback of the method is that it is too much work to construct an LR parser by hand for a typical programming-language grammar. One needs a specialized tool – an LR parser generator. Fortunately, many such generators are available, and we shall discuss the design and use of one, Yacc, in Section 4.9. With such a generator, one can write a context-free grammar and have the generator automatically produce a parser for that grammar. If the grammar contains ambiguities or other constructs that are difficult to parse in a left-to-right scan of the input, then the parser generator can locate these constructs and inform the compiler designer of their presence.

After discussing the operation of an LR parser, we present three techniques for constructing an LR parsing table for a grammar. The first method, called simple LR (SLR for short), is the easiest to implement, but the least powerful of the three. It may fail to produce a parsing table for certain grammars on which the other methods succeed. The second method, called canonical LR, is the most powerful, and the most expensive. The third method, called look-ahead LR (LALR for short), is intermediate in power and cost between the other two. The LALR method will work on most programming-language grammars and, with some effort, can be implemented efficiently. Some techniques for compressing the size of an LR parsing table are considered later in this section.

The LR Parsing Algorithm

The schematic form of an LR parser is shown in Fig. 4.29. It consists of an input, an output, a stack, a driver program, and a parsing table that has two parts (*action* and *goto*). The driver program is the same for all LR parsers; only the parsing table changes from one parser to another. The parsing program reads characters from an input buffer one at a time. The program uses a stack to store a string of the form $s_0X_1s_1X_2s_2 \cdots X_ms_m$, where s_m is on top. Each X_i is a grammar symbol and each s_i is a symbol called a *state*. Each state symbol summarizes the information contained in the stack below it, and the combination of the state symbol on top of the stack and the current input symbol are used to index the parsing table and determine the shift-reduce parsing decision. In an implementation, the grammar symbols need not appear on the stack; however, we shall always include them in our discussions to help explain the behavior of an LR parser.

The parsing table consists of two parts, a parsing action function *action* and a goto function *goto*. The program driving the LR parser behaves as follows. It determines s_m , the state currently on top of the stack, and a_i , the current input symbol. It then consults $action[s_m, a_i]$, the parsing action table entry for state s_m and input a_i , which can have one of four values:

1. shift s , where s is a state,
2. reduce by a grammar production $A \rightarrow \beta$,
3. accept, and
4. error.

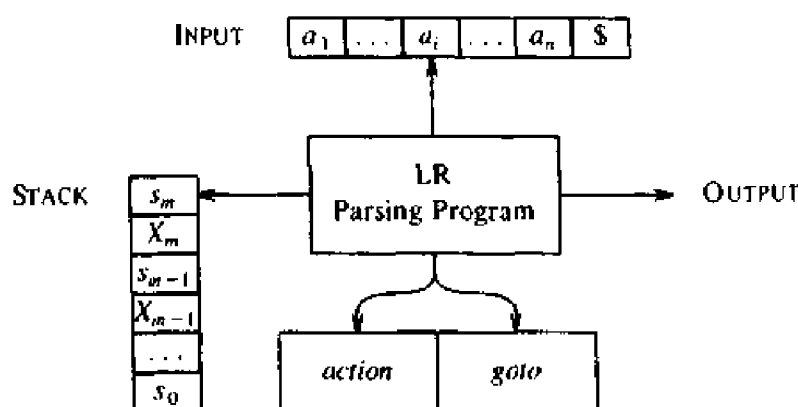


Fig. 4.29. Model of an LR parser.

The function *goto* takes a state and grammar symbol as arguments and produces a state. We shall see that the *goto* function of a parsing table constructed from a grammar G using the SLR, canonical LR, or LALR method is the transition function of a deterministic finite automaton that recognizes the viable prefixes of G . Recall that the viable prefixes of G are those prefixes of right-sentential forms that can appear on the stack of a shift-reduce parser, because they do not extend past the rightmost handle. The initial state of this DFA is the state initially put on top of the LR parser stack.

A *configuration* of an LR parser is a pair whose first component is the stack contents and whose second component is the unexpended input:

$$(s_0 X_1 s_1 X_2 s_2 \cdots X_m s_m, a_i a_{i+1} \cdots a_n \$)$$

This configuration represents the right-sentential form

$$X_1 X_2 \cdots X_m a_i a_{i+1} \cdots a_n$$

in essentially the same way as a shift-reduce parser would; only the presence of states on the stack is new.

The next move of the parser is determined by reading a_i , the current input symbol, and s_m , the state on top of the stack, and then consulting the parsing action table entry $action[s_m, a_i]$. The configurations resulting after each of the four types of move are as follows:

1. If $action[s_m, a_i] = \text{shift } s$, the parser executes a shift move, entering the configuration

$$(s_0 X_1 s_1 X_2 s_2 \cdots X_m s_m a_i s, a_{i+1} \cdots a_n \$)$$

Here the parser has shifted both the current input symbol a_i and the next state s , which is given in $action[s_m, a_i]$, onto the stack; a_{i+1} becomes the current input symbol.

2. If $\text{action}[s_m, a_i] = \text{reduce } A \rightarrow \beta$, then the parser executes a reduce move, entering the configuration

$$(s_0 X_1 s_1 X_2 s_2 \cdots X_{m-r} s_{m-r} A s, a_i a_{i+1} \cdots a_n \$)$$

where $s = \text{goto}[s_{m-r}, A]$ and r is the length of β , the right side of the production. Here the parser first popped $2r$ symbols off the stack (r state symbols and r grammar symbols), exposing state s_{m-r} . The parser then pushed both A , the left side of the production, and s , the entry for $\text{goto}[s_{m-r}, A]$, onto the stack. The current input symbol is not changed in a reduce move. For the LR parsers we shall construct, $X_{m-r+1} \cdots X_m$, the sequence of grammar symbols popped off the stack, will always match β , the right side of the reducing production.

The output of an LR parser is generated after a reduce move by executing the semantic action associated with the reducing production. For the time being, we shall assume the output consists of just printing the reducing production.

3. If $\text{action}[s_m, a_i] = \text{accept}$, parsing is completed.
4. If $\text{action}[s_m, a_i] = \text{error}$, the parser has discovered an error and calls an error recovery routine.

The LR parsing algorithm is summarized below. All LR parsers behave in this fashion; the only difference between one LR parser and another is the information in the parsing action and goto fields of the parsing table.

Algorithm 4.7. LR parsing algorithm.

Input. An input string w and an LR parsing table with functions *action* and *goto* for a grammar G .

Output. If w is in $L(G)$, a bottom-up parse for w ; otherwise, an error indication.

Method. Initially, the parser has s_0 on its stack, where s_0 is the initial state, and $w\$$ in the input buffer. The parser then executes the program in Fig. 4.30 until an accept or error action is encountered. $\square$

Example 4.33. Figure 4.31 shows the parsing action and goto functions of an LR parsing table for the following grammar for arithmetic expressions with binary operators $+$ and $*$:

- (1) $E \rightarrow E + T$
- (2) $E \rightarrow T$
- (3) $T \rightarrow T * F$
- (4) $T \rightarrow F$
- (5) $F \rightarrow (E)$
- (6) $F \rightarrow \text{id}$

The codes for the actions are:

```
set ip to point to the first symbol of w$;  
repeat forever begin  
  let s be the state on top of the stack and  
  a the symbol pointed to by ip;  
  if action[s, a] = shift s' then begin  
    push a then s' on top of the stack;  
    advance ip to the next input symbol  
  end  
  else if action[s, a] = reduce  $A \rightarrow \beta$  then begin  
    pop  $2 * |\beta|$  symbols off the stack;  
    let s' be the state now on top of the stack;  
    push A then goto[s', A] on top of the stack;  
    output the production  $A \rightarrow \beta$   
  end  
  else if action[s, a] = accept then  
    return  
  else error()  
end
```

Fig. 4.30. LR parsing program.

STATE	action						goto		
	id	+	*	(	)	\$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

Fig. 4.31. Parsing table for expression grammar.

- 1. *si* means shift and stack state *i*,
- 2. *rj* means reduce by production numbered *j*,
- 3. *acc* means accept,
- 4. blank means error.

Note that the value of $\text{goto}[s, a]$ for terminal a is found in the action field connected with the shift action on input a for state s . The goto field gives $\text{goto}[s, A]$ for nonterminals A . Also, bear in mind that we have not yet explained how the entries for Fig. 4.31 were selected; we shall deal with this issue shortly.

On input $\text{id} * \text{id} + \text{id}$, the sequence of stack and input contents is shown in Fig. 4.32. For example, at line (1) the LR parser is in state 0 with id the first input symbol. The action in row 0 and column id of the action field of Fig. 4.31 is $s5$, meaning shift and cover the stack with state 5. That is what has happened at line (2): the first token id and the state symbol 5 have both been pushed onto the stack, and id has been removed from the input.

Then, $*$ becomes the current input symbol, and the action of state 5 on input $*$ is to reduce by $F \rightarrow \text{id}$. Two symbols are popped off the stack (one state symbol and one grammar symbol). State 0 is then exposed. Since the goto of state 0 on F is 3, F and 3 are pushed onto the stack. We now have the configuration in line (3). Each of the remaining moves is determined similarly. $\square$

	STACK	INPUT	ACTION
(1)	0	$\text{id} * \text{id} + \text{id} \$$	shift
(2)	0 id 5	$* \text{id} + \text{id} \$$	reduce by $F \rightarrow \text{id}$
(3)	0 F 3	$* \text{id} + \text{id} \$$	reduce by $T \rightarrow F$
(4)	0 T 2	$* \text{id} + \text{id} \$$	shift
(5)	0 T 2 $*$ 7	$\text{id} + \text{id} \$$	shift
(6)	0 T 2 $*$ 7 id 5	$+ \text{id} \$$	reduce by $F \rightarrow \text{id}$
(7)	0 T 2 $*$ 7 F 10	$+ \text{id} \$$	reduce by $T \rightarrow T * F$
(8)	0 T 2	$+ \text{id} \$$	reduce by $E \rightarrow T$
(9)	0 E 1	$+ \text{id} \$$	shift
(10)	0 E 1 $+$ 6	$\text{id} \$$	shift
(11)	0 E 1 $+$ 6 id 5	$\$$	reduce by $F \rightarrow \text{id}$
(12)	0 E 1 $+$ 6 F 3	$\$$	reduce by $T \rightarrow F$
(13)	0 E 1 $+$ 6 T 9	$\$$	$E \rightarrow E + T$
(14)	0 E 1	$\$$	accept

Fig. 4.32. Moves of LR parser on $\text{id} * \text{id} + \text{id}$.

LR Grammars

How do we construct an LR parsing table for a given grammar? A grammar for which we can construct a parsing table is said to be an *LR grammar*. There are context-free grammars that are not LR, but these can generally be avoided for typical programming-language constructs. Intuitively, in order for a grammar to be LR it is sufficient that a left-to-right shift-reduce parser be able to recognize handles when they appear on top of the stack.

An LR parser does not have to scan the entire stack to know when the handle appears on top. Rather, the state symbol on top of the stack contains all the information it needs. It is a remarkable fact that if it is possible to recognize a handle knowing only the grammar symbols on the stack, then there is a finite automaton that can, by reading the grammar symbols on the stack from top to bottom, determine what handle, if any, is on top of the stack. The goto function of an LR parsing table is essentially such a finite automaton. The automaton need not, however, read the stack on every move. The state symbol stored on top of the stack is the state the handle-recognizing finite automaton would be in if it had read the grammar symbols of the stack from bottom to top. Thus, the LR parser can determine from the state on top of the stack everything that it needs to know about what is in the stack.

Another source of information that an LR parser can use to help make its shift-reduce decisions is the next k input symbols. The cases $k=0$ or $k=1$ are of practical interest, and we shall only consider LR parsers with $k \leq 1$ here. For example, the action table in Fig. 4.31 uses one symbol of lookahead. A grammar that can be parsed by an LR parser examining up to k input symbols on each move is called an *LR(k) grammar*.

There is a significant difference between LL and LR grammars. For a grammar to be LR(k), we must be able to recognize the occurrence of the right side of a production, having seen all of what is derived from that right side with k input symbols of lookahead. This requirement is far less stringent than that for LL(k) grammars where we must be able to recognize the use of a production seeing only the first k symbols of what its right side derives. Thus, LR grammars can describe more languages than LL grammars.

Constructing SLR Parsing Tables

We now show how to construct from a grammar an LR parsing table. We shall give three methods, varying in their power and ease of implementation. The first, called "simple LR" or *SLR* for short, is the weakest of the three in terms of the number of grammars for which it succeeds, but is the easiest to implement. We shall refer to the parsing table constructed by this method as an SLR table, and to an LR parser using an SLR parsing table as an SLR parser. A grammar for which an SLR parser can be constructed is said to be an SLR grammar. The other two methods augment the SLR method with lookahead information, so the SLR method is a good starting point for studying LR parsing.

An *LR(0) item* (item for short) of a grammar G is a production of G with a dot at some position of the right side. Thus, production $A \rightarrow XYZ$ yields the four items

$$\begin{aligned} A &\rightarrow \cdot XYZ \\ A &\rightarrow X \cdot YZ \\ A &\rightarrow XY \cdot Z \\ A &\rightarrow XYZ \cdot \end{aligned}$$

The production $A \rightarrow \epsilon$ generates only one item, $A \rightarrow \cdot$. An item can be represented by a pair of integers, the first giving the number of the production and the second the position of the dot. Intuitively, an item indicates how much of a production we have seen at a given point in the parsing process. For example, the first item above indicates that we hope to see a string derivable from XYZ next on the input. The second item indicates that we have just seen on the input a string derivable from X and that we hope next to see a string derivable from YZ .

The central idea in the SLR method is first to construct from the grammar a deterministic finite automaton to recognize viable prefixes. We group items together into sets, which give rise to the states of the SLR parser. The items can be viewed as the states of an NFA recognizing viable prefixes, and the "grouping together" is really the subset construction discussed in Section 3.6.

One collection of sets of LR(0) items, which we call the *canonical LR(0) collection*, provides the basis for constructing SLR parsers. To construct the canonical LR(0) collection for a grammar, we define an augmented grammar and two functions, *closure* and *goto*.

If G is a grammar with start symbol S , then G' , the *augmented grammar* for G , is G with a new start symbol S' and production $S' \rightarrow S$. The purpose of this new starting production is to indicate to the parser when it should stop parsing and announce acceptance of the input. That is, acceptance occurs when and only when the parser is about to reduce by $S' \rightarrow S$.

The Closure Operation

If I is a set of items for a grammar G , then $\text{closure}(I)$ is the set of items constructed from I by the two rules:

1. Initially, every item in I is added to $\text{closure}(I)$.
2. If $A \rightarrow \alpha \cdot B \beta$ is in $\text{closure}(I)$ and $B \rightarrow \gamma$ is a production, then add the item $B \rightarrow \cdot \gamma$ to I , if it is not already there. We apply this rule until no more new items can be added to $\text{closure}(I)$.

Intuitively, $A \rightarrow \alpha \cdot B \beta$ in $\text{closure}(I)$ indicates that, at some point in the parsing process, we think we might next see a substring derivable from $B\beta$ as input. If $B \rightarrow \gamma$ is a production, we also expect we might see a substring derivable from γ at this point. For this reason we also include $B \rightarrow \cdot \gamma$ in $\text{closure}(I)$.

Example 4.34. Consider the augmented expression grammar:

$$\begin{aligned} E' &\rightarrow E \\ E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow (E) \mid \text{id} \end{aligned} \tag{4.19}$$

If I is the set of one item $\{[E' \rightarrow \cdot E]\}$, then $\text{closure}(I)$ contains the items

$$\begin{aligned}
 E' &\rightarrow \cdot E \\
 E &\rightarrow \cdot E + T \\
 E &\rightarrow \cdot T \\
 T &\rightarrow \cdot T * F \\
 T &\rightarrow \cdot F \\
 F &\rightarrow \cdot (E) \\
 F &\rightarrow \cdot \text{id}
 \end{aligned}$$

Here, $E' \rightarrow \cdot E$ is put in $\text{closure}(I)$ by rule (1). Since there is an E immediately to the right of a dot, by rule (2) we add the E -productions with dots at the left end, that is, $E \rightarrow \cdot E + T$ and $E \rightarrow \cdot T$. Now there is a T immediately to the right of a dot, so we add $T \rightarrow \cdot T * F$ and $T \rightarrow \cdot F$. Next, the F to the right of a dot forces $F \rightarrow \cdot (E)$ and $F \rightarrow \cdot \text{id}$ to be added. No other items are put into $\text{closure}(I)$ by rule (2). $\square$

The function closure can be computed as in Fig. 4.33. A convenient way to implement the function closure is to keep a boolean array *added*, indexed by the nonterminals of G , such that $\text{added}[B]$ is set to true if and when we add the items $B \rightarrow \cdot \gamma$ for each B -production $B \rightarrow \gamma$.

```

function closure ( I );
begin
    J := I;
    repeat
        for each item  $A \rightarrow \alpha \cdot B \beta$  in J and each production
             $B \rightarrow \gamma$  of G such that  $B \rightarrow \cdot \gamma$  is not in J do
            add  $B \rightarrow \cdot \gamma$  to J
    until no more items can be added to J;
    return J
end

```

Fig. 4.33. Computation of *closure*.

Note that if one B -production is added to the closure of I with the dot at the left end, then all B -productions will be similarly added to the closure. In fact, it is not necessary in some circumstances actually to list the items $B \rightarrow \cdot \gamma$ added to I by closure . A list of the nonterminals B whose productions were so added will suffice. In fact, it turns out that we can divide all the sets of items we are interested in into two classes of items.

1. *Kernel items*, which include the initial item, $S' \rightarrow \cdot S$, and all items whose dots are not at the left end.
2. *Nonkernel items*, which have their dots at the left end.

Moreover, each set of items we are interested in is formed by taking the closure of a set of kernel items; the items added in the closure can never be

kernel items, of course. Thus, we can represent the sets of items we are really interested in with very little storage if we throw away all nonkernel items, knowing that they could be regenerated by the closure process.

The Goto Operation

The second useful function is $\text{goto}(I, X)$ where I is a set of items and X is a grammar symbol. $\text{goto}(I, X)$ is defined to be the closure of the set of all items $[A \rightarrow \alpha X \beta]$ such that $[A \rightarrow \alpha \cdot X \beta]$ is in I . Intuitively, if I is the set of items that are valid for some viable prefix γ , then $\text{goto}(I, X)$ is the set of items that are valid for the viable prefix γX .

Example 4.35. If I is the set of two items $\{[E' \rightarrow E \cdot], [E \rightarrow E \cdot + T]\}$, then $\text{goto}(I, +)$ consists of

$$\begin{aligned} E &\rightarrow E + \cdot T \\ T &\rightarrow \cdot T * F \\ T &\rightarrow \cdot F \\ F &\rightarrow \cdot (E) \\ F &\rightarrow \cdot \text{id} \end{aligned}$$

We computed $\text{goto}(I, +)$ by examining I for items with $+$ immediately to the right of the dot. $E' \rightarrow E \cdot$ is not such an item, but $E \rightarrow E \cdot + T$ is. We moved the dot over the $+$ to get $\{E \rightarrow E + \cdot T\}$ and then took the closure of this set. $\square$

The Sets-of-Items Construction

We are now ready to give the algorithm to construct C , the canonical collection of sets of LR(0) items for an augmented grammar G' ; the algorithm is shown in Fig. 4.34.

```

procedure items( $G'$ );
begin
     $C := \{\text{closure}(\{[S' \rightarrow \cdot S]\})\}$ ;
    repeat
        for each set of items  $I$  in  $C$  and each grammar symbol  $X$ 
            such that  $\text{goto}(I, X)$  is not empty and not in  $C$  do
                add  $\text{goto}(I, X)$  to  $C$ 
    until no more sets of items can be added to  $C$ 
end

```

Fig. 4.34. The sets-of-items construction.

Example 4.36. The canonical collection of sets of LR(0) items for grammar (4.19) of Example 4.34 is shown in Fig. 4.35. The goto function for this set of items is shown as the transition diagram of a deterministic finite automaton D in Fig. 4.36. $\square$

$I_0:$	$E' \rightarrow \cdot E$ $E \rightarrow \cdot E + T$ $E \rightarrow \cdot T$ $T \rightarrow \cdot T * F$ $T \rightarrow \cdot F$ $F \rightarrow \cdot (E)$ $F \rightarrow \cdot id$	$I_5:$	$F \rightarrow id \cdot$
$I_1:$	$E' \rightarrow E \cdot$ $E \rightarrow E \cdot + T$	$I_6:$	$E \rightarrow E + \cdot T$ $T \rightarrow \cdot T * F$ $T \rightarrow \cdot F$ $F \rightarrow \cdot (E)$ $F \rightarrow \cdot id$
$I_2:$	$E \rightarrow T \cdot$ $T \rightarrow T \cdot * F$	$I_7:$	$T \rightarrow T * \cdot F$ $F \rightarrow \cdot (E)$ $F \rightarrow \cdot id$
$I_3:$	$T \rightarrow F \cdot$	$I_8:$	$F \rightarrow (E \cdot)$ $E \rightarrow E \cdot + T$
$I_4:$	$F \rightarrow (\cdot E)$ $E \rightarrow \cdot E + T$ $E \rightarrow \cdot T$ $T \rightarrow \cdot T * F$ $T \rightarrow \cdot F$ $F \rightarrow \cdot (E)$ $F \rightarrow \cdot id$	$I_9:$	$E \rightarrow E + T \cdot$ $T \rightarrow T \cdot * F$
		$I_{10}:$	$T \rightarrow T * F \cdot$
		$I_{11}:$	$F \rightarrow (E) \cdot$

Fig. 4.35. Canonical LR(0) collection for grammar (4.19).

If each state of D in Fig. 4.36 is a final state and I_0 is the initial state, then D recognizes exactly the viable prefixes of grammar (4.19). This is no accident. For every grammar G , the *goto* function of the canonical collection of sets of items defines a deterministic finite automaton that recognizes the viable prefixes of G . In fact, one can visualize a nondeterministic finite automaton N whose states are the items themselves. There is a transition from $A \rightarrow \alpha \cdot X \beta$ to $A \rightarrow \alpha X \cdot \beta$ labeled X , and there is a transition from $A \rightarrow \alpha \cdot B \beta$ to $B \rightarrow \cdot \gamma$ labeled ϵ . Then $\text{closure}(I)$ for set of items (states of N) I is exactly the ϵ -closure of a set of NFA states defined in Section 3.6. Thus, $\text{goto}(I, X)$ gives the transition from I on symbol X in the DFA constructed from N by the subset construction. Viewed in this way, the procedure $\text{items}(G')$ in Fig. 4.34 is just the subset construction itself applied to the NFA N constructed from G' as we have described.

Valid items. We say item $A \rightarrow \beta_1 \cdot \beta_2$ is *valid* for a viable prefix $\alpha \beta_1$ if there is a derivation $S' \xRightarrow{*}_{rm} \alpha A w \xRightarrow{*}_{rm} \alpha \beta_1 \beta_2 w$. In general, an item will be valid for many viable prefixes. The fact that $A \rightarrow \beta_1 \cdot \beta_2$ is valid for $\alpha \beta_1$, tells us a lot about whether to shift or reduce when we find $\alpha \beta_1$ on the parsing stack. In

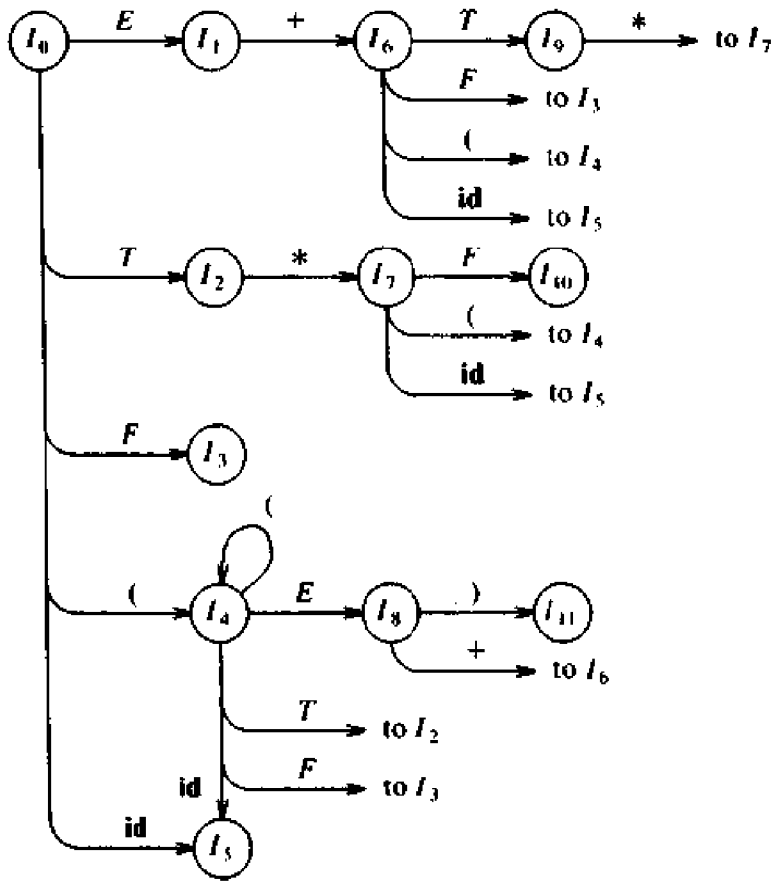


Fig. 4.36. Transition diagram of DFA D for viable prefixes.

particular, if $\beta_2 \neq \epsilon$, then it suggests that we have not yet shifted the handle onto the stack, so shift is our move. If $\beta_2 = \epsilon$, then it looks as if $A \rightarrow \beta_1$ is the handle, and we should reduce by this production. Of course, two valid items may tell us to do different things for the same viable prefix. Some of these conflicts can be resolved by looking at the next input symbol, and others can be resolved by the methods of the next section, but we should not suppose that all parsing action conflicts can be resolved if the LR method is used to construct a parsing table for an arbitrary grammar.

We can easily compute the set of valid items for each viable prefix that can appear on the stack of an LR parser. In fact, it is a central theorem of LR parsing theory that the set of valid items for a viable prefix γ is exactly the set of items reached from the initial state along a path labeled γ in the DFA constructed from the canonical collection of sets of items with transitions given by *goto*. In essence, the set of valid items embodies all the useful information that can be gleaned from the stack. While we shall not prove this theorem here, we shall give an example.

Example 4.37. Let us consider the grammar (4.19) again, whose sets of items

and *goto* function are exhibited in Fig. 4.35 and 4.36. Clearly, the string $E + T *$ is a viable prefix of (4.19). The automaton of Fig. 4.36 will be in state I_7 after having read $E + T *$. State I_7 contains the items

$$T \rightarrow T * \cdot F$$

$$F \rightarrow \cdot (E)$$

$$F \rightarrow \cdot id$$

which are precisely the items valid for $E + T *$. To see this, consider the following three rightmost derivations

$$E' \Rightarrow E$$

$$\Rightarrow E + T$$

$$\Rightarrow E + T * F$$

$$E' \Rightarrow E$$

$$\Rightarrow E + T$$

$$\Rightarrow E + T * F$$

$$\Rightarrow E + T * (E)$$

$$E' \Rightarrow E$$

$$\Rightarrow E + T$$

$$\Rightarrow E + T * F$$

$$\Rightarrow E + T * id$$

The first derivation shows the validity of $T \rightarrow T * \cdot F$, the second the validity of $F \rightarrow \cdot (E)$, and the third the validity of $F \rightarrow \cdot id$ for the viable prefix $E + T *$. It can be shown that there are no other valid items for $E + T *$, and we leave a proof to the interested reader. $\square$

SLR Parsing Tables

Now we shall show how to construct the SLR parsing action and *goto* functions from the deterministic finite automaton that recognizes viable prefixes. Our algorithm will not produce uniquely defined parsing action tables for all grammars, but it does succeed on many grammars for programming languages. Given a grammar, G , we augment G to produce G' , and from G' we construct C , the canonical collection of sets of items for G' . We construct *action*, the parsing action function, and *goto*, the *goto* function, from C using the following algorithm. It requires us to know FOLLOW(A) for each nonterminal A of a grammar (see Section 4.4).

Algorithm 4.8. Constructing an SLR parsing table.

Input. An augmented grammar G' .

Output. The SLR parsing table functions *action* and *goto* for G' .

Method.

1. Construct $C = \{I_0, I_1, \dots, I_n\}$, the collection of sets of LR(0) items for G' .
2. State i is constructed from I_i . The parsing actions for state i are determined as follows:
 - a) If $[A \rightarrow \alpha \cdot a \beta]$ is in I_i and $\text{goto}(I_i, a) = I_j$, then set $\text{action}[i, a]$ to "shift j ." Here a must be a terminal.
 - b) If $[A \rightarrow \alpha \cdot]$ is in I_i , then set $\text{action}[i, a]$ to "reduce $A \rightarrow \alpha$ " for all a

in FOLLOW(A); here A may not be S' .

- c) If $[S' \rightarrow S \cdot]$ is in I_i , then set $action[i, \$]$ to "accept."

If any conflicting actions are generated by the above rules, we say the grammar is not SLR(1). The algorithm fails to produce a parser in this case.

3. The goto transitions for state i are constructed for all nonterminals A using the rule: If $goto(I_i, A) = I_j$, then $goto[i, A] = j$.
4. All entries not defined by rules (2) and (3) are made "error."
5. The initial state of the parser is the one constructed from the set of items containing $[S' \rightarrow \cdot S]$. □

The parsing table consisting of the parsing action and goto functions determined by Algorithm 4.8 is called the *SLR(1) table for G* . An LR parser using the SLR(1) table for G is called the SLR(1) parser for G , and a grammar having an SLR(1) parsing table is said to be *SLR(1)*. We usually omit the "(1)" after the "SLR," since we shall not deal here with parsers having more than one symbol of lookahead.

Example 4.38. Let us construct the SLR table for grammar (4.19). The canonical collection of sets of LR(0) items for (4.19) was shown in Fig. 4.35. First consider the set of items I_0 :

$$\begin{aligned} E' &\rightarrow \cdot E \\ E &\rightarrow \cdot E + T \\ E &\rightarrow \cdot T \\ T &\rightarrow \cdot T * F \\ T &\rightarrow \cdot F \\ F &\rightarrow \cdot (E) \\ F &\rightarrow \cdot id \end{aligned}$$

The item $F \rightarrow \cdot (E)$ gives rise to the entry $action[0, ()] = \text{shift 4}$, the item $F \rightarrow \cdot id$ to the entry $action[0, id] = \text{shift 5}$. Other items in I_0 yield no actions. Now consider I_1 :

$$\begin{aligned} E' &\rightarrow E \cdot \\ E &\rightarrow E \cdot + T \end{aligned}$$

The first item yields $action[1, \$] = \text{accept}$, the second yields $action[1, +] = \text{shift 6}$. Next consider I_2 :

$$\begin{aligned} E &\rightarrow T \cdot \\ T &\rightarrow T \cdot * F \end{aligned}$$

Since FOLLOW(E) = $\{\$, +,)\}$, the first item makes $action[2, \$] = action[2, +] = action[2,)] = \text{reduce } E \rightarrow T$. The second item makes $action[2, *] = \text{shift 7}$. Continuing in this fashion we obtain the parsing action and goto tables that were shown in Fig. 4.31. In that figure, the numbers of productions in reduce actions are the same as the order in which they appear

in the original grammar (4.18). That is, $E \rightarrow E+T$ is number 1. $E \rightarrow T$ is 2, and so on. $\square$

Example 4.39. Every SLR(1) grammar is unambiguous, but there are many unambiguous grammars that are not SLR(1). Consider the grammar with productions

$$\begin{aligned} S &\rightarrow L = R \\ S &\rightarrow R \\ L &\rightarrow * R \\ L &\rightarrow \text{id} \\ R &\rightarrow L \end{aligned} \tag{4.20}$$

We may think of L and R as standing for l -value and r -value, respectively, and $*$ as an operator indicating “contents of.”³ The canonical collection of sets of LR(0) items for grammar (4.20) is shown in Fig. 4.37.

$$\begin{array}{ll} I_0: & S' \rightarrow \cdot S \\ & S \rightarrow \cdot L = R \\ & S \rightarrow \cdot R \\ & L \rightarrow \cdot * R \\ & L \rightarrow \cdot \text{id} \\ & R \rightarrow \cdot L \\ I_1: & S' \rightarrow S \cdot \\ I_2: & S \rightarrow L \cdot = R \\ & R \rightarrow L \cdot \\ I_3: & S \rightarrow R \cdot \\ I_4: & L \rightarrow * \cdot R \\ & R \rightarrow \cdot L \\ & L \rightarrow * \cdot R \\ & L \rightarrow * \text{id} \\ I_5: & L \rightarrow \text{id} \cdot \\ I_6: & S \rightarrow L = \cdot R \\ & R \rightarrow \cdot L \\ & L \rightarrow * \cdot R \\ & L \rightarrow * \text{id} \\ I_7: & L \rightarrow * R \cdot \\ I_8: & R \rightarrow L \cdot \\ I_9: & S \rightarrow L = R \cdot \end{array}$$

Fig. 4.37. Canonical LR(0) collection for grammar (4.20).

Consider the set of items I_2 . The first item in this set makes $action[2, =]$ be “shift 6.” Since $FOLLOW(R)$ contains $=$, (to see why, consider $S \Rightarrow L = R \Rightarrow * R = R$), the second item sets $action[2, =]$ to “reduce $R \rightarrow L$.” Thus entry $action[2, =]$ is multiply defined. Since there is both a shift and a reduce entry in $action[2, =]$, state 2 has a shift/reduce conflict on

³ As in Section 2.8, an l -value designates a location and an r -value is a value that can be stored in a location.

input symbol =.

Grammar (4.20) is not ambiguous. This shift/reduce conflict arises from the fact that the SLR parser construction method is not powerful enough to remember enough left context to decide what action the parser should take on input = having seen a string reducible to L . The canonical and LALR methods, to be discussed next, will succeed on a larger collection of grammars, including grammar (4.20). It should be pointed out, however, that there are unambiguous grammars for which every LR parser construction method will produce a parsing action table with parsing action conflicts. Fortunately, such grammars can generally be avoided in programming language applications. $\square$

Constructing Canonical LR Parsing Tables

We shall now present the most general technique for constructing an LR parsing table from a grammar. Recall that in the SLR method, state i calls for reduction by $A \rightarrow \alpha$ if the set of items I_i contains item $[A \rightarrow \alpha \cdot]$ and a is in $\text{FOLLOW}(A)$. In some situations, however, when state i appears on top of the stack, the viable prefix $\beta\alpha$ on the stack is such that βA cannot be followed by a in a right-sentential form. Thus, the reduction by $A \rightarrow \alpha$ would be invalid on input a .

Example 4.40. Let us reconsider Example 4.39, where in state 2 we had item $R \rightarrow L \cdot$, which could correspond to $A \rightarrow \alpha$ above, and a could be the = sign, which is in $\text{FOLLOW}(R)$. Thus, the SLR parser calls for reduction by $R \rightarrow L$ in state 2 with = as the next input (the shift action is also called for because of item $S \rightarrow L \cdot = R$ in state 2). However, there is no right-sentential form of the grammar in Example 4.39 that begins $R = \dots$. Thus state 2, which is the state corresponding to viable prefix L only, should not really call for reduction of that L to R . $\square$

It is possible to carry more information in the state that will allow us to rule out some of these invalid reductions by $A \rightarrow \alpha$. By splitting states when necessary, we can arrange to have each state of an LR parser indicate exactly which input symbols can follow a handle α for which there is a possible reduction to A .

The extra information is incorporated into the state by redefining items to include a terminal symbol as a second component. The general form of an item becomes $[A \rightarrow \alpha \cdot \beta, a]$, where $A \rightarrow \alpha\beta$ is a production and a is a terminal or the right endmarker \$. We call such an object an *LR(l) item*. The l refers to the length of the second component, called the *lookahead* of the item.⁴ The lookahead has no effect in an item of the form $[A \rightarrow \alpha \cdot \beta, a]$, where β is not ϵ , but an item of the form $[A \rightarrow \alpha \cdot, a]$ calls for a reduction by $A \rightarrow \alpha$ only if

⁴ Lookaheads that are strings of length greater than one are possible, of course, but we shall not consider such lookaheads here.

the next input symbol is a . Thus, we are compelled to reduce by $A \rightarrow \alpha$ only on those input symbols a for which $[A \rightarrow \alpha, a]$ is an LR(1) item in the state on top of the stack. The set of such a 's will always be a subset of $\text{FOLLOW}(A)$, but it could be a proper subset, as in Example 4.40.

Formally, we say LR(1) item $[A \rightarrow \alpha\beta, a]$ is *valid* for a viable prefix γ if there is a derivation $S \xRightarrow{*}_{rm} \delta Aw \xRightarrow{rm} \delta\alpha\beta w$, where

1. $\gamma = \delta\alpha$, and
2. either a is the first symbol of w , or w is ϵ and a is $\$$.

Example 4.41. Let us consider the grammar

$$\begin{aligned} S &\rightarrow BB \\ B &\rightarrow aB \mid b \end{aligned}$$

There is a rightmost derivation $S \xRightarrow{*}_{rm} aaBab \xRightarrow{rm} aaaBab$. We see that item $[B \rightarrow aB, a]$ is valid for a viable prefix $\gamma = aaa$ by letting $\delta = aa$, $A = B$, $w = ab$, $\alpha = a$, and $\beta = B$ in the above definition.

There is also a rightmost derivation $S \xRightarrow{*}_{rm} BaB \xRightarrow{rm} BaaB$. From this derivation we see that item $[B \rightarrow aB, \$]$ is valid for viable prefix Baa . $\square$

The method for constructing the collection of sets of valid LR(1) items is essentially the same as the way we built the canonical collection of sets of LR(0) items. We only need to modify the two procedures *closure* and *goto*.

To appreciate the new definition of the *closure* operation, consider an item of the form $[A \rightarrow \alpha B\beta, a]$ in the set of items valid for some viable prefix γ . Then there is a rightmost derivation $S \xRightarrow{*}_{rm} \delta Aax \xRightarrow{rm} \delta\alpha B\beta ax$, where $\gamma = \delta\alpha$. Suppose βax derives terminal string by . Then for each production of the form $B \rightarrow \eta$ for some η , we have derivation $S \xRightarrow{*}_{rm} \gamma Bby \xRightarrow{rm} \gamma\eta by$. Thus, $[B \rightarrow \eta, b]$ is valid for γ . Note that b can be the first terminal derived from β , or it is possible that β derives ϵ in the derivation $\beta ax \xRightarrow{*} by$, and b can therefore be a . To summarize both possibilities we say that b can be any terminal in $\text{FIRST}(\beta ax)$, where FIRST is the function from Section 4.4. Note that x cannot contain the first terminal of by , so $\text{FIRST}(\beta ax) = \text{FIRST}(\beta a)$. We now give the LR(1) sets of items construction.

Algorithm 4.9. Construction of the sets of LR(1) items.

Input. An augmented grammar G' .

Output. The sets of LR(1) items that are the set of items valid for one or more viable prefixes of G' .

Method. The procedures *closure* and *goto* and the main routine *items* for constructing the sets of items are shown in Fig. 4.38. $\square$

Example 4.42. Consider the following augmented grammar.

$$\begin{aligned} S' &\rightarrow S \\ S &\rightarrow CC \\ C &\rightarrow cC \mid d \end{aligned} \tag{4.21}$$

```

function closure(I);
begin
    repeat
        for each item [ $A \rightarrow \alpha \cdot B \beta$ ,  $a$ ] in I,
            each production  $B \rightarrow \gamma$  in  $G'$ ,
            and each terminal  $b$  in  $\text{FIRST}(\beta a)$ 
            such that  $[B \rightarrow \cdot \gamma, b]$  is not in I do
                add  $[B \rightarrow \cdot \gamma, b]$  to I;
    until no more items can be added to I;
    return I
end;

function goto(I, X);
begin
    let J be the set of items [ $A \rightarrow \alpha X \cdot \beta$ ,  $a$ ] such that
        [ $A \rightarrow \alpha \cdot X \beta$ ,  $a$ ] is in I;
    return closure(J)
end;

procedure items( $G'$ );
begin
     $C := \{\text{closure}(\{[S' \rightarrow \cdot S, \$])\}$ ;
    repeat
        for each set of items I in C and each grammar symbol X
            such that goto(I, X) is not empty and not in C do
                add goto(I, X) to C
    until no more sets of items can be added to C
end

```

Fig. 4.38. Sets of LR(1) items construction for grammar G' .

We begin by computing the closure of $\{[S' \rightarrow \cdot S, \$]\}$. To close, we match the item $[S' \rightarrow \cdot S, \$]$ with the item $[A \rightarrow \alpha \cdot B \beta, a]$ in the procedure *closure*. That is, $A = S'$, $\alpha = \epsilon$, $B = S$, $\beta = \epsilon$, and $a = \$$. Function *closure* tells us to add $[B \rightarrow \cdot \gamma, b]$ for each production $B \rightarrow \gamma$ and terminal b in $\text{FIRST}(\beta a)$. In terms of the present grammar, $B \rightarrow \gamma$ must be $S \rightarrow CC$, and since β is ϵ and a is $\$, b$ may only be $\$$. Thus we add $[S \rightarrow \cdot CC, \$]$.

We continue to compute the closure by adding all items $[C \rightarrow \cdot \gamma, b]$ for b in $\text{FIRST}(C\$)$. That is, matching $[S \rightarrow \cdot CC, \$]$ against $[A \rightarrow \alpha \cdot B \beta, a]$ we have $A = S$, $\alpha = \epsilon$, $B = C$, $\beta = \epsilon$, and $a = \$$. Since C does not derive the empty string, $\text{FIRST}(C\$) = \text{FIRST}(C)$. Since $\text{FIRST}(C)$ contains terminals c and d , we add items $[C \rightarrow \cdot cC, c]$, $[C \rightarrow \cdot cC, d]$, $[C \rightarrow \cdot d, c]$ and $[C \rightarrow \cdot d, d]$. None of the new items has a nonterminal immediately to the right of the dot, so we have completed our first set of LR(1) items. The initial set of items is:

$$\begin{aligned}
 I_0: \quad & S' \rightarrow \cdot S, \$ \\
 & S \rightarrow \cdot CC, \$ \\
 & C \rightarrow \cdot cC, c/d \\
 & C \rightarrow \cdot d, c/d
 \end{aligned}$$

The brackets have been omitted for notational convenience, and we use the notation $[C \rightarrow \cdot cC, c/d]$ as a shorthand for the two items $[C \rightarrow \cdot cC, c]$ and $[C \rightarrow \cdot cC, d]$.

Now we compute $\text{goto}(I_0, X)$ for the various values of X . For $X = S$ we must close the item $[S' \rightarrow S \cdot, \$]$. No additional closure is possible, since the dot is at the right end. Thus we have the next set of items:

$$I_1: \quad S' \rightarrow S \cdot, \$$$

For $X = C$ we close $[S \rightarrow C \cdot C, \$]$. We add the C -productions with second component $\$$ and then can add no more, yielding:

$$\begin{aligned}
 I_2: \quad & S \rightarrow C \cdot C, \$ \\
 & C \rightarrow \cdot cC, \$ \\
 & C \rightarrow \cdot d, \$
 \end{aligned}$$

Next, let $X = c$. We must close $\{[C \rightarrow c \cdot C, c/d]\}$. We add the C -productions with second component c/d , yielding:

$$\begin{aligned}
 I_3: \quad & C \rightarrow c \cdot C, c/d \\
 & C \rightarrow \cdot cC, c/d \\
 & C \rightarrow \cdot d, c/d
 \end{aligned}$$

Finally, let $X = d$, and we wind up with the set of items:

$$I_4: \quad C \rightarrow d \cdot, c/d$$

We have finished considering goto on I_0 . We get no new sets from I_1 , but I_2 has goto 's on C , c , and d . On C we get:

$$I_5: \quad S \rightarrow CC \cdot, \$$$

no closure being needed. On c we take the closure of $\{[C \rightarrow c \cdot C, \$]\}$, to obtain:

$$\begin{aligned}
 I_6: \quad & C \rightarrow c \cdot C, \$ \\
 & C \rightarrow \cdot cC, \$ \\
 & C \rightarrow \cdot d, \$
 \end{aligned}$$

Note that I_6 differs from I_3 only in second components. We shall see that it is common for several sets of LR(1) items for a grammar to have the same first components and differ in their second components. When we construct the collection of sets of LR(0) items for the same grammar, each set of LR(0) items will coincide with the set of first components of one or more sets of LR(1) items. We shall have more to say about this phenomenon when we discuss LALR parsing.

Continuing with the goto function for I_2 , $\text{goto}(I_2, d)$ is seen to be:

$$I_7: C \rightarrow d \cdot, \$$$

Turning now to I_3 , the *goto*'s of I_3 on c and d are I_3 and I_4 , respectively, and *goto*(I_3 , C) is:

$$I_8: C \rightarrow cC \cdot, c/d$$

I_4 and I_5 have no *goto*'s. The *goto*'s of I_6 on c and d are I_6 and I_7 , respectively, and *goto*(I_6 , C) is:

$$I_9: C \rightarrow cC \cdot, \$$$

The remaining sets of items yield no *goto*'s, so we are done. Figure 4.39 shows the ten sets of items with their *goto*'s. $\square$

We now give the rules whereby the LR(1) parsing action and *goto* functions are constructed from the sets of LR(1) items. The action and *goto* functions are represented by a table as before. The only difference is in the values of the entries.

Algorithm 4.10. Construction of the canonical LR parsing table.

Input. An augmented grammar G' .

Output. The canonical LR parsing table functions *action* and *goto* for G' .

Method.

1. Construct $C = \{I_0, I_1, \dots, I_n\}$, the collection of sets of LR(1) items for G' .
2. State i of the parser is constructed from I_i . The parsing actions for state i are determined as follows:
 - a) If $[A \rightarrow \alpha a \beta, b]$ is in I_i and *goto*(I_i , a) = I_j , then set *action*[i , a] to "shift j ." Here, a is required to be a terminal.
 - b) If $[A \rightarrow \alpha \cdot, a]$ is in I_i , $A \neq S'$, then set *action*[i , a] to "reduce $A \rightarrow \alpha$."
 - c) If $[S' \rightarrow S \cdot, \$]$ is in I_i , then set *action*[i , $\$$] to "accept."

If a conflict results from the above rules, the grammar is said not to be LR(1), and the algorithm is said to fail.

3. The *goto* transitions for state i are determined as follows: If *goto*(I_i , A) = I_j , then *goto*[i , A] = j .
4. All entries not defined by rules (2) and (3) are made "error."
5. The initial state of the parser is the one constructed from the set containing item $[S' \rightarrow \cdot S, \$]$. $\square$

The table formed from the parsing action and *goto* functions produced by Algorithm 4.10 is called the *canonical* LR(1) parsing table. An LR parser using this table is called a canonical LR(1) parser. If the parsing action

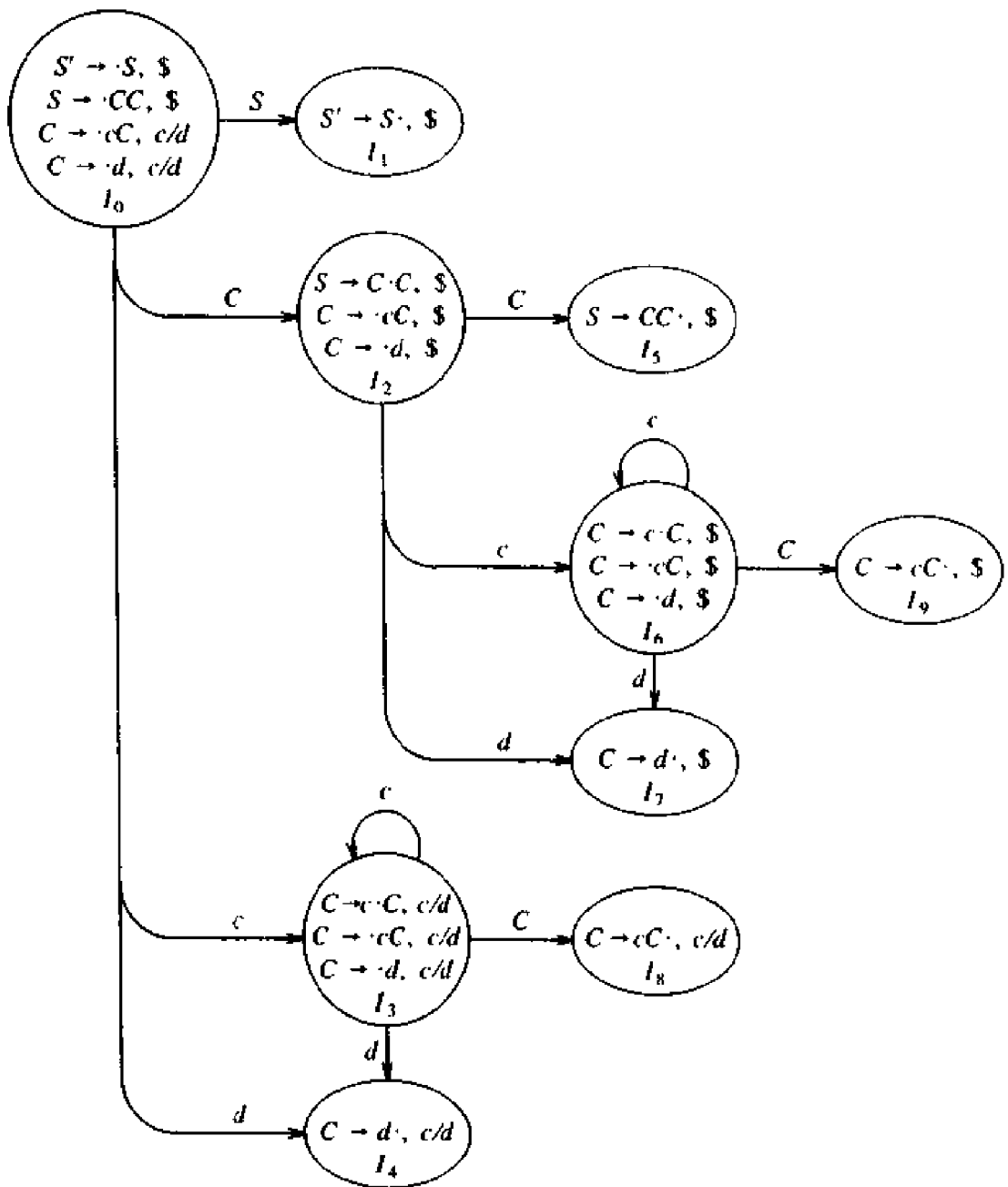


Fig. 4.39. The *goto* graph for grammar (4.21).

function has no multiply-defined entries, then the given grammar is called an *LR(1) grammar*. As before, we omit the “(1)” if it is understood.

Example 4.43. The canonical parsing table for the grammar (4.21) is shown in Fig. 4.40. Productions 1, 2, and 3 are $S \rightarrow CC$, $C \rightarrow cC$, and $C \rightarrow d$. $\square$

Every *SLR(1)* grammar is an *LR(1)* grammar, but for an *SLR(1)* grammar the canonical LR parser may have more states than the *SLR* parser for the

STATE	action			goto	
	c	d	\$	S	C
0	s3	s4		1	2
1			acc		
2	s6	s7			5
3	s3	s4			8
4	r3	r3			
5			r1		
6	s6	s7			9
7			r3		
8	r2	r2			
9			r2		

Fig. 4.40. Canonical parsing table for grammar (4.21).

same grammar. The grammar of the previous examples is SLR and has an SLR parser with seven states, compared with the ten of Fig. 4.40.

Constructing LALR Parsing Tables

We now introduce our last parser construction method, the LALR (*lookahead-LR*) technique. This method is often used in practice because the tables obtained by it are considerably smaller than the canonical LR tables, yet most common syntactic constructs of programming languages can be expressed conveniently by an LALR grammar. The same is almost true for SLR grammars, but there are a few constructs that cannot be conveniently handled by SLR techniques (see Example 4.39, for example).

For a comparison of parser size, the SLR and LALR tables for a grammar always have the same number of states, and this number is typically several hundred states for a language like Pascal. The canonical LR table would typically have several thousand states for the same size language. Thus, it is much easier and more economical to construct SLR and LALR tables than the canonical LR tables.

By way of introduction, let us again consider the grammar (4.21), whose sets of LR(1) items were shown in Fig. 4.39. Take a pair of similar looking states, such as I_4 and I_7 . Each of these states has only items with first component $C \rightarrow d$. In I_4 , the lookaheads are c or d ; in I_7 , $\$$ is the only lookahead.

To see the difference between the roles of I_4 and I_7 in the parser, note that grammar (4.21) generates the regular set c^*dc^*d . When reading an input $cc \cdots cdcc \cdots cd$, the parser shifts the first group of c 's and their following d onto the stack, entering state 4 after reading the d . The parser then calls for a reduction by $C \rightarrow d$, provided the next input symbol is c or d . The

requirement that c or d follow makes sense, since these are the symbols that could begin strings in c^*d . If $\$$ follows the first d , we have an input like ccd , which is not in the language, and state 4 correctly declares an error if $\$$ is the next input.

The parser enters state 7 after reading the second d . Then, the parser must see $\$$ on the input, or it started with a string not of the form c^*dc^*d . It thus makes sense that state 7 should reduce by $C \rightarrow d$ on input $\$$ and declare error on inputs c or d .

Let us now replace I_4 and I_7 by I_{47} , the union of I_4 and I_7 , consisting of the set of three items represented by $[C \rightarrow d \cdot, c/d/\$]$. The goto's on d to I_4 or I_7 from I_0, I_2, I_3 , and I_6 now enter I_{47} . The action of state 47 is to reduce on any input. The revised parser behaves essentially like the original, although it might reduce d to C in circumstances where the original would declare error, for example, on input like ccd or $cdcdc$. The error will eventually be caught; in fact, it will be caught before any more input symbols are shifted.

More generally, we can look for sets of LR(1) items having the same *core*, that is, set of first components, and we may merge these sets with common cores into one set of items. For example, in Fig. 4.39, I_4 and I_7 form such a pair, with core $\{C \rightarrow d \cdot\}$. Similarly, I_3 and I_6 form another pair, with core $\{C \rightarrow c \cdot C, C \rightarrow \cdot cC, C \rightarrow \cdot d\}$. There is one more pair, I_8 and I_9 , with core $\{C \rightarrow cC \cdot\}$. Note that, in general, a core is a set of LR(0) items for the grammar at hand, and that an LR(1) grammar may produce more than two sets of items with the same core.

Since the core of $\text{goto}(I, X)$ depends only on the core of I , the goto's of merged sets can themselves be merged. Thus, there is no problem revising the goto function as we merge sets of items. The action functions are modified to reflect the non-error actions of all sets of items in the merger.

Suppose we have an LR(1) grammar, that is, one whose sets of LR(1) items produce no parsing action conflicts. If we replace all states having the same core with their union, it is possible that the resulting union will have a conflict, but it is unlikely for the following reason: Suppose in the union there is a conflict on lookahead a because there is an item $[A \rightarrow \alpha \cdot, a]$ calling for a reduction by $A \rightarrow \alpha$, and there is another item $[B \rightarrow \beta \cdot a\gamma, b]$ calling for a shift. Then some set of items from which the union was formed has item $[A \rightarrow \alpha \cdot, a]$, and since the cores of all these states are the same, it must have an item $[B \rightarrow \beta \cdot a\gamma, c]$ for some c . But then this state has the same shift/reduce conflict on a , and the grammar was not LR(1) as we assumed. Thus, the merging of states with common cores can never produce a shift/reduce conflict that was not present in one of the original states, because shift actions depend only on the core, not the lookahead.

It is possible, however, that a merger will produce a reduce/reduce conflict, as the following example shows.

Example 4.44. Consider the grammar

$$\begin{aligned}
 S' &\rightarrow S \\
 S &\rightarrow aAd \mid bBd \mid aBe \mid bAe \\
 A &\rightarrow c \\
 B &\rightarrow c
 \end{aligned}$$

which generates the four strings acd , ace , bcd , and bce . The reader can check that the grammar is LR(1) by constructing the sets of items. Upon doing so, we find the set of items $\{[A \rightarrow c \cdot, d], [B \rightarrow c \cdot, e]\}$ valid for viable prefix ac and $\{[A \rightarrow c \cdot, e], [B \rightarrow c \cdot, d]\}$ valid for bc . Neither of these sets generates a conflict, and their cores are the same. However, their union, which is

$$\begin{aligned}
 A &\rightarrow c \cdot, d/e \\
 B &\rightarrow c \cdot, d/e
 \end{aligned}$$

generates a reduce/reduce conflict, since reductions by both $A \rightarrow c$ and $B \rightarrow c$ are called for on inputs d and e . $\square$

We are now prepared to give the first of two LALR table construction algorithms. The general idea is to construct the sets of LR(1) items, and if no conflicts arise, merge sets with common cores. We then construct the parsing table from the collection of merged sets of items. The method we are about to describe serves primarily as a definition of LALR(1) grammars. Constructing the entire collection of LR(1) sets of items requires too much space and time to be useful in practice.

Algorithm 4.11. An easy, but space-consuming LALR table construction.

Input. An augmented grammar G' .

Output. The LALR parsing table functions *action* and *goto* for G' .

Method.

1. Construct $C = \{I_0, I_1, \dots, I_n\}$, the collection of sets of LR(1) items.
2. For each core present among the set of LR(1) items, find all sets having that core, and replace these sets by their union.
3. Let $C' = \{J_0, J_1, \dots, J_m\}$ be the resulting sets of LR(1) items. The parsing actions for state i are constructed from J_i in the same manner as in Algorithm 4.10. If there is a parsing action conflict, the algorithm fails to produce a parser, and the grammar is said not to be LALR(1).
4. The *goto* table is constructed as follows. If J is the union of one or more sets of LR(1) items, that is, $J = I_1 \cup I_2 \cup \dots \cup I_k$, then the cores of $\text{goto}(I_1, X)$, $\text{goto}(I_2, X)$, $\dots$, $\text{goto}(I_k, X)$ are the same, since $I_1, I_2, \dots, I_k$ all have the same core. Let K be the union of all sets of items having the same core as $\text{goto}(I_1, X)$. Then $\text{goto}(J, X) = K$. $\square$

The table produced by Algorithm 4.11 is called the *LALR parsing table* for G . If there are no parsing action conflicts, then the given grammar is said to

be an *LALR(1) grammar*. The collection of sets of items constructed in step (3) is called the *LALR(1) collection*.

Example 4.45. Again consider the grammar (4.21) whose *goto* graph was shown in Fig. 4.39. As we mentioned, there are three pairs of sets of items that can be merged. I_3 and I_6 are replaced by their union:

$$\begin{aligned} I_{36}: \quad & C \rightarrow c \cdot C, c/d/\$ \\ & C \rightarrow \cdot cC, c/d/\$ \\ & C \rightarrow \cdot d, c/d/\$ \end{aligned}$$

I_4 and I_7 are replaced by their union:

$$I_{47}: \quad C \rightarrow d \cdot, c/d/\$$$

and I_8 and I_9 are replaced by their union:

$$I_{89}: \quad C \rightarrow cC \cdot, c/d/\$$$

The LALR action and *goto* functions for the condensed sets of items are shown in Fig. 4.41.

STATE	action			goto	
	c	d	\$	S	C
0	s36	s47		1	2
1			acc		
2	s36	s47			5
36	s36	s47			89
47	r3	r3	r3		
5			r1		
89	r2	r2	r2		

Fig. 4.41. LALR parsing table for grammar (4.21).

To see how the *goto*'s are computed, consider $\text{goto}(I_{36}, C)$. In the original set of LR(1) items, $\text{goto}(I_3, C) = I_8$, and I_8 is now part of I_{89} , so we make $\text{goto}(I_{36}, C)$ be I_{89} . We could have arrived at the same conclusion if we considered I_6 , the other part of I_{36} . That is, $\text{goto}(I_6, C) = I_9$, and I_9 is now part of I_{89} . For another example, consider $\text{goto}(I_2, c)$, an entry that is exercised after the shift action of I_2 on input c . In the original sets of LR(1) items, $\text{goto}(I_2, c) = I_6$. Since I_6 is now part of I_{36} , $\text{goto}(I_2, c)$ becomes I_{36} . Thus, the entry in Fig. 4.41 for state 2 and input c is made s36, meaning shift and push state 36 onto the stack. $\square$

When presented with a string from the language c^*dc^*d , both the LR parser of Fig. 4.40 and the LALR parser of Fig. 4.41 make exactly the same sequence of shifts and reductions, although the names of the states on the stack may differ; i.e., if the LR parser puts I_3 or I_6 on the stack, the LALR

parser will put I_{36} on the stack. This relationship holds in general for an LALR grammar. The LR and LALR parsers will mimic one another on correct inputs.

However, when presented with erroneous input, the LALR parser may proceed to do some reductions after the LR parser has declared an error, although the LALR parser will never shift another symbol after the LR parser declares an error. For example, on input ccd followed by $\$$, the LR parser of Fig. 4.34 will put

$0\ c\ 3\ c\ 3\ d\ 4$

on the stack, and in state 4 will discover an error, because $\$$ is the next input symbol and state 4 has action error on $\$$. In contrast, the LALR parser of Fig. 4.41 will make the corresponding moves, putting

$0\ c\ 36\ c\ 36\ d\ 47$

on the stack. But state 47 on input $\$$ has action reduce $C \rightarrow d$. The LALR parser will thus change its stack to

$0\ c\ 36\ c\ 36\ C\ 89$

Now the action of state 89 on input $\$$ is reduce $C \rightarrow cC$. The stack becomes

$0\ c\ 36\ C\ 89$

whereupon a similar reduction is called for, obtaining stack

$0\ C\ 2$

Finally, state 2 has action error on input $\$$, so the error is now discovered. $\square$

Efficient Construction of LALR Parsing Tables

There are several modifications we can make to Algorithm 4.11 to avoid constructing the full collection of sets of LR(1) items in the process of creating an LALR(1) parsing table. The first observation is that we can represent a set of items I by its kernel, that is, by those items that are either the initial item $[S' \rightarrow \cdot S, \$]$, or that have the dot somewhere other than at the beginning of the right side.

Second, we can compute the parsing actions generated by I from the kernel alone. Any item calling for a reduction by $A \rightarrow \alpha$ will be in the kernel unless $\alpha = \epsilon$. Reduction by $A \rightarrow \epsilon$ is called for on input a if and only if there is a kernel item $[B \rightarrow \gamma \cdot C\delta, b]$ such that $C \xRightarrow{*}_{rm} A\eta$ for some η , and a is in $\text{FIRST}(\eta\delta b)$. The set of nonterminals A such that $C \xRightarrow{*}_{rm} A\eta$ can be precomputed for each nonterminal C .

The shift actions generated by I can be determined from the kernel of I as follows. We shift on input a if there is a kernel item $[B \rightarrow \gamma \cdot C\delta, b]$ where $C \xRightarrow{*}_{rm} a\alpha$ in a derivation in which the last step does not use an ϵ -production. The set of such a 's can also be precomputed for each C .

Here is how the goto transitions for I can be computed from the kernel. If

$[B \rightarrow \gamma X \delta, b]$ is in the kernel of I , then $[B \rightarrow \gamma X \cdot \delta, b]$ is in the kernel of $\text{goto}(I, X)$. Item $[A \rightarrow X \cdot \beta, a]$ is also in the kernel of $\text{goto}(I, X)$ if there is an item $[B \rightarrow \gamma C \delta, b]$ in the kernel of I , and $C \xRightarrow{*}_{rm} A\eta$ for some η . If we precompute for each pair of nonterminals C and A whether $C \xRightarrow{*}_{rm} A\eta$ for some η , then computing sets of items from kernels only is just slightly less efficient than doing so with closed sets of items.

To compute the LALR(1) sets of items for an augmented grammar G' , we start with the kernel $S' \rightarrow \cdot S$ of the initial set of items I_0 . Then, we compute the kernels of the goto transitions from I_0 as outlined above. We continue computing the goto transitions for each new kernel generated until we have the kernels of the entire collection of sets of LR(0) items.

Example 4.46. Let us again consider the augmented grammar

$$\begin{aligned} S' &\rightarrow S \\ S &\rightarrow L = R \mid R \\ L &\rightarrow * R \mid \text{id} \\ R &\rightarrow L \end{aligned}$$

The kernels of the sets of LR(0) items for this grammar are shown in Fig. 4.42. $\square$

$I_0: S' \rightarrow \cdot S$	$I_5: L \rightarrow \text{id} \cdot$
$I_1: S' \rightarrow S \cdot$	$I_6: S \rightarrow L = \cdot R$
$I_2: S \rightarrow L = \cdot R$ $R \rightarrow L \cdot$	$I_7: L \rightarrow * \cdot R$
$I_3: S \rightarrow R \cdot$	$I_8: R \rightarrow L \cdot$
$I_4: L \rightarrow * \cdot R$	$I_9: S \rightarrow L = R \cdot$

Fig. 4.42. Kernels of the sets of LR(0) items for grammar (4.20).

Now we expand the kernels by attaching to each LR(0) item the proper lookahead symbols (second components). To see how lookahead symbols propagate from a set of items I to $\text{goto}(I, X)$, consider an LR(0) item $B \rightarrow \gamma C \delta$ in the kernel of I . Suppose $C \xRightarrow{*}_{rm} A\eta$ for some η (perhaps $C = A$ and $\eta = \epsilon$), and $A \rightarrow X\beta$ is a production. Then LR(0) item $A \rightarrow X \cdot \beta$ is in $\text{goto}(I, X)$.

Suppose now that we are computing not LR(0) items, but LR(1) items, and $[B \rightarrow \gamma C \delta, b]$ is in the set I . Then for what values of a will $[A \rightarrow X \cdot \beta, a]$ be in $\text{goto}(I, X)$? Certainly if some a is in $\text{FIRST}(\eta\delta)$, then the derivation $C \xRightarrow{*}_{rm} A\eta$ tells us that $[A \rightarrow X \cdot \beta, a]$ must be in $\text{goto}(I, X)$. In this case, the value of b is irrelevant, and we say that a , as a lookahead for $A \rightarrow X \cdot \beta$, is generated *spontaneously*. By definition, $\$$ is generated spontaneously as a lookahead for the item $S' \rightarrow \cdot S$ in the initial set of items.

But there is another source of lookaheads for item $A \rightarrow X \cdot \beta$. If $\eta\delta \xRightarrow{*} \epsilon$,

then $[A \rightarrow X \cdot \beta, b]$ will also be in $\text{goto}(I, X)$. We say, in this case, that look-aheads *propagate* from $B \rightarrow \gamma \cdot C \delta$ to $A \rightarrow X \cdot \beta$. A simple method to determine when an LR(1) item in I generates a lookahead in $\text{goto}(I, X)$ spontaneously, and when lookaheads propagate, is contained in the next algorithm.

Algorithm 4.12. Determining lookaheads.

Input. The kernel K of a set of LR(0) items I and a grammar symbol X .

Output. The lookaheads spontaneously generated by items in I for kernel items in $\text{goto}(I, X)$ and the items in I from which lookaheads are propagated to kernel items in $\text{goto}(I, X)$.

Method. The algorithm is given in Fig. 4.43. It uses a dummy lookahead symbol $\#$ to detect situations in which lookaheads propagate. $\square$

```

for each item  $B \rightarrow \gamma \cdot \delta$  in  $K$  do begin
     $J' := \text{closure}(\{[B \rightarrow \gamma \cdot \delta, \#]\})$ ;
    if  $[A \rightarrow \alpha \cdot X \beta, a]$  is in  $J'$  where  $a$  is not  $\#$  then
        lookahead  $a$  is generated spontaneously for item
             $A \rightarrow \alpha \cdot X \beta$  in  $\text{goto}(I, X)$ ;
    if  $[A \rightarrow \alpha \cdot X \beta, \#]$  is in  $J'$  then
        lookaheads propagate from  $B \rightarrow \gamma \cdot \delta$  in  $I$  to
             $A \rightarrow \alpha \cdot X \beta$  in  $\text{goto}(I, X)$ 
end

```

Fig. 4.43. Discovering propagated and spontaneous lookaheads.

Now let us consider how we go about finding the lookaheads associated with the items in the kernels of the sets of LR(0) items. First, we know that $\$$ is a lookahead for $S' \rightarrow \cdot S$ in the initial set of LR(0) items. Algorithm 4.12 gives us all the lookaheads generated spontaneously. After listing all those lookaheads, we must allow them to propagate until no further propagation is possible. There are many different approaches, all of which in some sense keep track of "new" lookaheads that have propagated to an item but which have not yet propagated out. The next algorithm describes one technique to propagate lookaheads to all items.

Algorithm 4.13. Efficient computation of the kernels of the LALR(1) collection of sets of items.

Input. An augmented grammar G' .

Output. The kernels of the LALR(1) collection of sets of items for G' .

Method.

1. Using the method outlined above, construct the kernels of the sets of LR(0) items for G .